

國立陽明交通大學
電機資訊國際學位學程
碩士論文

EECS International Graduate Program
National Yang Ming Chiao Tung University
Master Thesis

基於人工智慧的工業控制系統跨層級攻擊偵測與應變
AI-Based Cross-Level Attack Detection and Response
in Industrial Control System

研 究 生： 範庭貴（Pham Dinh Quy）

指導教授： 林盈達（Ying-Dar Lin）

中華民國一一四年六月

June 2026

基於人工智慧的工業控制系統跨層級攻擊偵測與應變
AI-Based Cross-Level Attack Detection and Response
in Industrial Control System

研 究 生：範庭貴
指導教授：林盈達 博士

Student: Pham Dinh Quy
Advisor: Dr. Ying-Dar Lin

國立陽明交通大學
電機資訊國際學位學程
碩士論文

A Thesis

Submitted to EECS International Graduate Program
College of Electrical Engineering and Computer Science
National Yang Ming Chiao Tung University
in Partial Fulfilment of the Requirements
for the Degree of
Master in
Electrical Engineering and Computer Science

June 2026

Taiwan, Republic of China

中華民國 一一四年六月

基於人工智慧的工業控制系統跨層級攻擊偵測與應變

學生：範庭貴

指導教授：林盈達 博士

國立陽明交通大學 電機資訊國際學位學程

摘 要

工業控制系統 (Industrial Control Systems, ICS) 通常跨越多個 OT 層級運作，主要包括 Level 2 (監控控制層) 與 Level 1 (實體控制層)，兩者透過控制命令與感測器資料緊密耦合。此種耦合特性使跨層級攻擊 (Cross-Level Attack) 成為可能，即攻擊跡象可能出現在 L2，而其實體後果則於 L1 顯現，導致安全營運中心 (SOC) 難以有效偵測攻擊並選擇適當的回應措施。然而，目前大多數研究仍聚焦於單一層級監控，忽略了此類跨層級互動。為了解決上述問題，本研究提出一套適用於 ICS 的多層級 AI Agent 架構，可同時針對 OT 控制層 (L1-L2) 進行自動化偵測與回應。該系統結合非監督式學習 (Unsupervised Learning, UL) 進行異常偵測、Transformer 用於攻擊技術分類，以及強化學習 (Reinforcement Learning, RL) 用於動態回應策略選擇。本研究於基於 Docker 的 ICS 測試平台中實作所提出之架構，模擬水處理流程，並蒐集日誌 (logs)、網路流量 (network traffic) 與感測器資料 (sensor data) 作為分析來源。實驗結果顯示，在相同攻擊情境下，Cross-Level 架構於偵測與回應兩方面皆穩定優於單層級基準模型。在偵測方面，Cross-Level 架構達到最高的 macro F1-score (0.96)，而最佳的 Single-Level 基準模型在 Level 2 與 Level 1 僅分別達到約 0.87 與 0.81。在回應方面，Cross-Level Agent 維持約 80% 的高成功率，而 Single-Level 的 L2 與 L1 Agent 則僅分別達到約 50% 與 20%。此外，Cross-Level Agent 在所有攻擊階段中皆展現出穩定的強健性 (robustness)；相較之下，L2 Agent 常於 kill chain 後期失敗，而 L1 Agent 則經常於早期與中期階段失敗。上述結果顯示，Cross-Level AI Agent 能在 ICS 環境中提供更精確的攻擊偵測、更深入的攻擊理解，以及更有效的自動化回應能力，為未來自主化與自適應 OT 安全防護提供一個具實務可行性的方向。

關鍵詞：工業控制系統 (ICS/OT)、跨層級攻擊 (Cross-Level Attack)、跨層級偵測與回應 (Cross-Level Detection & Response)、非監督式學習 (Unsupervised Learning)、Transformer、強化學習 (Reinforcement Learning)

AI-Based Cross-Level Attack Detection and Response in Industrial Control System

Student: Pham Dinh Quy

Advisor: Dr. Ying-Dar Lin

EECS International Graduate Program
National Yang Ming Chiao Tung University

Abstract

Industrial Control Systems (ICS) operate across OT levels, mainly Level 2 - L2 (supervisory control) and Level 1 -L1 (physical control), which are tightly coupled through commands and sensor data. This coupling enables cross-level attacks, where the footprint appears at L2 while the physical consequences manifest at L1, making it difficult for SOC's to detect and select appropriate response actions. Yet most existing work still focuses on single-level monitoring and overlooks such cross-level interactions. To address this gap, we propose a multi-level AI agent architecture for ICS that automatically detects and responds across both OT Control Level (L1-L2). The system combines Unsupervised Learning (UL) for anomaly detection, Transformer for classifying attack techniques, and Reinforcement Learning (RL) for dynamic response selection. We implement the proposed framework in a Docker-based ICS testbed simulating a water treatment process, with logs, network traffic, and sensor data. Experimental results show that under the same attack scenarios, the Cross-Level architecture consistently outperforms single-level baselines in both detection and response. For detection, it reaches the highest macro F1 (0.96), while the best Single-Level baselines achieve only around 0.87 at Level 2 and 0.81 at Level 1. For response, the Cross-Level agent maintains a high success rate of approximately 80%, whereas the Single-Level L2 and L1 agents reach only about 50% and 20%, respectively. Moreover, while the Cross-Level agent remains robust across all attack stages, the L2 agent often fails in the later stages of kill chain and the L1 agent frequently fails in the early and middle stages. These results demonstrate that Cross-Level AI agents can provide more accurate detection, deeper attack understanding, and more effective automated response in ICS environments, offering a practical direction toward autonomous and adaptive OT security.

Keywords: Industrial Control Systems (ICS/OT), Cross Level Attack, Cross Level Detection & Response, Unsupervised Learning, Transformer, Reinforcement Learning.

Acknowledgement

回顧這段人生旅程，我滿懷感激之情。這篇論文的完成從來不是一條輕鬆的道路，途中充滿了疑惑、不眠之夜，以及幾度幾乎想要放棄的時刻。然而，在這些困難的時刻推動我繼續前行的，是他人的支持與善意。

首先，我要向我的指導教授林盈達老師表達我最深的感謝。您對「為什麼」的不斷追問，不僅塑造了我研究的邏輯，更深刻地影響了我面對問題的思考方式。您教會我如何更精確、更有耐心地探索問題，並保有好奇心。這些學習將超越論文本身，伴隨我未來的道路。我也誠摯感謝賴源正老師與黃仁竑老師，您們在定期討論中所提供的指導與建議，讓我不斷改進研究內容，並體會到學術清晰與嚴謹的重要性。我特別感謝我的中文老師林佳慧教授，您不僅是出色的教師，更是我真摯的朋友與個人靈感的來源。您溫暖的教學風格與不吝鼓勵讓每堂課都意義非凡。謝謝您一直相信我。

感謝 701 MIRC 實驗室的所有夥伴們——您們不僅是同事，更是陪我一起走過這段旅程的重要存在。我要特別感謝學長。您們的指導、見解與正能量對我意義非凡。也謝謝所有的室友與學弟妹們，感謝你們的歡笑、討論，以及那份共同堅持的心。這些回憶我將永遠珍藏於心中。

我也要感謝我的家人。即使相隔千里，您們每週的來電與不曾中斷的支持，始終是我最堅實的後盾。每一句鼓勵的話語都在我最困難的時候響起，給予我力量。謝謝我親愛的女友淳云，感謝你無盡的愛、理解與耐心。你總是在我低落時鼓勵我，提醒我自己的力量，並陪我慶祝每一個小小的成就。在我風雨飄搖之中，你的存在總能讓我平靜下來。感謝分布在世界各地的摯友們，我非常想念你們。即使我們相隔遙遠，你們的支持依然帶給我巨大的力量。謝謝你們讓我知道，我從不孤單。

最後，我要感謝自己。這是一段充滿挑戰的旅程，無論是學術、情緒或生活。我曾在實驗室與房間裡熬過無數個夜晚，靠著咖啡與那份默默的堅持撐過來。尤其在第一年，有過無數次想要放棄的時候，但我沒有放棄，我堅持下來，並完成了它。

這篇論文不僅是研究成果的體現，更是人與人之間情感、成長與韌性的結晶。我將你們的名字一一珍藏於此篇頁之中。

Table of Contents

摘要	i
Abstract	ii
Acknowledgement	iii
Table of Contents	iv
List of Figures	vi
List of Tables	vii
1 Introduction	1
2 Background and Related Works	4
2.1 ICS Architecture and Operational Characteristics	4
2.2 Threat Model and Attack Taxonomy in OT	6
2.3 AI in OT Security	8
2.4 Related Works on AI-based Detection and Response in ICS/OT	9
3 Problem Statement	13
3.1 Notations	13
3.2 Problem Overview	14
3.3 Problem Statement	16
4 Cross-Level Attack Detection and Response	18
4.1 Solution Overview - Agent Architecture	18
4.2 Attack Technique Detection	20
4.3 RL module for response	22
5 Implementation	26
5.1 Open Source Tools and Libraries	26
5.2 Implementation Details	27
5.2.1 Testbed	28
5.2.2 Implementation of AI Detection and Response Pipeline	31

6 Evaluation and Results	35
6.1 Experimental Setup & Performance Metrics	35
6.2 Cross Level vs Single Level	36
6.3 Detection Module Performance	42
6.4 State Space: Big vs Small	45
7 Conclusion and Future Work	48
7.1 Conclusion	48
7.2 Future Work	49
References	51

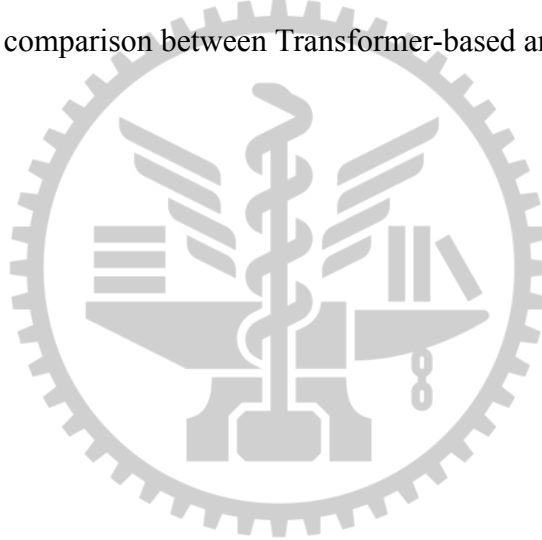


List of Figures

2.1	Purdue model for ICS Architecture [1]	5
2.2	MITRE ATT&CK Framework for ICS & Example Kill Chain	7
2.3	Reinforcement Learning	9
3.1	Example of Cross-Level Kill Chain in ICS-OT	15
3.2	Problem Setting of Cross-Level AI Agent in OT	16
4.1	Solution Overview	20
4.2	Technique Detection Solution	21
4.3	RL Module for Response	22
5.1	Implementation Overview	28
5.2	Example of Correlated Alert	33
6.1	Overall Detection Performance Across Agents	37
6.2	Confusion matrices of three agent configurations	38
6.3	Global Success rate over Training Episodes	39
6.4	Local Success Rate over Training Episodes	40
6.5	Attack Success Rate over Stage	41
6.6	Transformer vs LLM Performance Comparision	44
6.7	Confusion Matrices of the Anomaly Detection Module Across Multi-Source	45
6.8	Success Rate Comparison: Full State vs Small State	46
6.9	Reward Curve Comparison: Full State vs Small State	46
6.10	Action Distribution Comparison: Full State vs Small State	47

List of Tables

2.1	Related Works on AI-based Detection and Response in OT	10
3.1	Notation summary used in the proposed framework	14
4.1	Action table	23
5.1	Tools and Python libraries used in our implementation	27
5.2	Attack techniques used in the ICS testbed based on MITRE ATT&CK for ICS .	29
6.1	Q-Table Technique vs Actions	42
6.2	Performance comparison between Transformer-based and LLM-based classifiers	43



Chapter 1

Introduction

Industrial Control Systems (ICS) are cyber-physical platforms that manage critical physical processes in sectors such as energy, water treatment, and manufacturing. These systems operate physical process within the Operational Technology (OT) domain, which primarily consists of two control levels: Level 2 - The supervisory control level, where Human-Machine Interfaces (HMIs) allow operators to send out physical control commands; and Level 1, the physical control level, where Programmable Logic Controllers (PLCs) directly control physical devices such as sensors and actuators, then sending measurement data back to Level 2.

This bidirectional coupling between these two control levels unintentionally enables cross-level attacks, where an adversary acts at one level but causes consequences at another [2]. For example, under MITRE ATT&CK for ICS technique T0855 (Unauthorized Command Message) [3], if an attacker compromises HMIs at Level 2, they can send unauthorized commands to PLCs at Level 1. In such scenarios, the physical damage occurs at Level 1, but the primary attack footprint remains at Level 2, making it difficult for Security Operations Centers (SOCs) to detect the intrusion or give a proper action to respond effectively. detection and response — as two central stages in standard cyber-defense lifecycles [4] should not examine each level in isolation but should leverage context from both levels to understand what is actually happening and take appropriate actions to stop the attack.

Despite this need for cross-level awareness, most existing ICS security research pays little attention to this problem. In the area of detection, prior studies have applied various machine learning approaches to identify anomalies or classify attack types from OT data [5] [6]. Although some of these works collect data from multiple levels [7], they analyze each level's observations independently without examining how events at one level correlate with those at another. In cross-level attack scenarios, signals observed at individual levels are often ambiguous — a com-

mand at Level 2 may appear as a routine supervisory action, while a process deviation at Level 1 may be misattributed to an operational anomaly. Only by correlating signals across levels does the true attack picture emerge, allowing defenders to understand the overall situation and identify the attack more accurately. Without such cross-level correlation, defenders risk both missing the attack and misidentifying its nature.

Beyond detection, response is equally essential to prevent an ongoing attack from escalating further across OT levels. According to Fortinet 2024 State of Operational Technology and Cybersecurity report [8], OT organizations still struggle with manual incident response such as triaging alerts, correlating logs, applying containment changes,... Such workflows are often too slow compared to the pace at which modern attacks escalate across OT networks. This challenge has motivated several previous research exploring automated response using reinforcement learning (RL), which can learn adaptive policies that balance attack mitigation with process stability [9] [10]. However, existing RL-based response studies share two key limitations. First, they often rely on simplified detection modules [11], [12] limiting their ability to select precise mitigation actions. Second, the actions are typically operated within a single OT level without coordinating responses across levels [13], [14], [15]. Yet in cross-level attacks such as T0855 described above, effective mitigation requires simultaneous actions at multiple levels such as blocking the compromised HMIs at Level 2 then recovering the PLCs state at Level 1.

To address these gaps in both detection and response, this work proposes a unified AI-driven defense architecture that can analyze multi-source, multi-level OT data, detect and raise alerts for malicious activities, and select appropriate response actions for those detected alerts. The architecture combines a sequential pipeline of AI components. First, the UL model detects anomalous events on each OT data source across different levels. For every anomaly, we extract key features to form a short per-source alert, and then aggregate these alerts within a fixed time window into a unified cross-level alert. Then, a transformer-based classifier receives these cross-level alerts and predict the most likely attack technique. Finally, a reinforcement learning (RL) agent takes the identified technique and the current cross-level system state to select coordinated

response actions across the affected levels, balancing attack mitigation with process stability.

Our work makes three key contributions to Cross-Level attack detection and response in ICS/OT, addressing gaps identified in existing studies

1. We address the limitation of existing ICS security research that lacks cross-level reasoning and integrated detection-response modeling, where SOC's team relies on level-local observations and response is designed separately, failing to capture multi-stage attack
2. We propose an integrated AI-based framework for end-to-end cross-level detection and response, combining unsupervised learning for anomaly detection, Transformer for cross-level reasoning to classify ICS attack techniques, and reinforcement learning (RL) for automated response to prevent attack escalation.
3. We demonstrate the effectiveness of the proposed framework through evaluations in simulated ICS environments, showing that Cross-Level approaches improve attack detection, response coordination, and overall system stability compared to single-level baselines

The remainder of this thesis is organized as follows. Chapter 2 presents background on ICS/OT systems and surveys related work in anomaly detection, automated response in OT. Chapters 3 and 4 articulate the problem formulation and propose our solutions, respectively. Chapter 5 explains the implementation of the framework, while Chapter 6 reports the experimental setup, evaluation metrics, and results. Chapter 7 concludes the thesis and outlines directions for future research.

Chapter 2

Background and Related Works

This chapter first introduces the architecture and operational characteristics of industrial control systems (ICS), emphasizing how multi-level OT deployments and tight inter-level coupling shape both normal operations and cross-level attack surfaces. It then revisits the OT threat model and an ICS-specific attack taxonomy, highlighting adversary goals, capabilities, and kill-chain stages. Next, the chapter outlines the role of artificial intelligence in OT security and how data-driven methods can support monitoring, decision-making, and cross-level situational awareness under real-time, safety-critical constraints. Finally, it reviews prior work on AI-based detection and response in OT and identifies remaining gaps in end-to-end, cross-level defence that motivate the architecture proposed in this thesis.

2.1 ICS Architecture and Operational Characteristics

Industrial Control Systems (ICS) are commonly organized according to the Purdue Enterprise Reference Architecture, which structures components from enterprise Information Technology (IT) down to shop-floor Operational Technology (OT) [16]. As illustrate in Figure 2.1, in this model, the IT domain includes Level 4, corresponding to the enterprise network that hosts business and corporate applications, and Level 3, which handles site operations and production management. The OT domain comprises Levels 0-2: Level 2 provides supervisory control via Human-Machine Interfaces (HMIs) and Supervisory Control and Data Acquisition (SCADA) systems, Level 1 implements basic control through programmable logic controllers (PLCs), and Level 0 represents the physical process with sensors and actuators

Data and control flow through OT levels in a closed loop: operators use HMIs at Level 2 to send commands and setpoints to PLCs at Level 1, PLCs translate these into control signals that drive sensors and actuators at Level 0, and sensor measurements propagate back through the

PLCs to the HMIs to update process status and alarms. This architecture operates under strict real-time constraints, where timing and determinism are essential for safe control, and high availability is a primary objective because industrial processes are expected to run continuously with minimal downtime. ICS environments are also safety-critical, since faults or mis-operations can endanger equipment and human operators, and many devices in the OT levels are resource-constrained or legacy systems with limited computational and security capabilities [17].

In this thesis, we focus on the protection of the physical and control layers in the OT domain, where process manipulation can directly impact safety and availability. Although the Purdue model distinguishes between Level 0 (physical process) and Level 1 (basic control), these layers are tightly coupled in practice: PLC logic and field devices interact continuously in a single control loop and are often engineered and operated as one unit. To reflect this coupling and simplify our threat and defense model, we treat Levels 0 and 1 as a single protection domain, which we refer to as “Level 1” throughout the remainder of this thesis

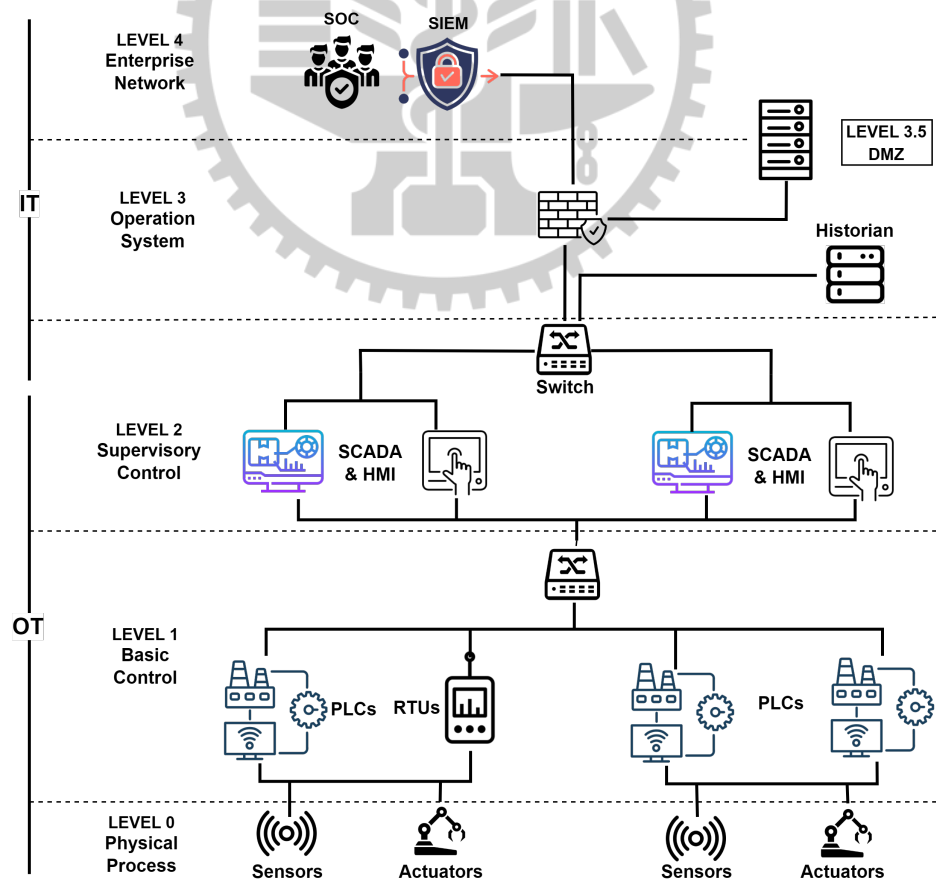


Figure 2.1: Purdue model for ICS Architecture [1]

2.2 Threat Model and Attack Taxonomy in OT

The integration of IT and OT extends the attack surface from enterprise systems to OT control networks. Attackers may enter through multiple entry points, including external vectors (e.g., phishing, exposed services), insider access, or remote connectivity such as VPN and vendor maintenance channels. A typical attack progression involves gaining an initial foothold in IT or remote access zones, followed by lateral movement toward the OT boundary, access to supervisory systems (HMI/SCADA), and ultimately manipulation of PLC logic or control commands, leading to physical process impact.

OT environments often exhibit weaker security postures due to legacy devices, limited segmentation, and infrequent patching. Combined with their real-time, continuous, and safety-critical nature, disruptions or malicious actions can result in significant operational and physical consequences. These characteristics, along with the high potential impact, make OT systems attractive targets for adversaries.

To systematically describe and categorize the attack behaviors outlined in the threat model, the MITRE ATT&CK for ICS framework as shown in Figure 2.2 provides a structured reference taxonomy. It defines a knowledge base of adversary tactics and techniques specific to industrial control environments, organized into a matrix where each tactic comprises multiple techniques that describe how attackers achieve their objectives across OT systems such as PLCs, HMIs, and engineering workstations.

This framework enables practitioners to map observed behaviors—such as host logs, network traffic, and process anomalies—to standardized technique identifiers, supporting threat-informed analysis and the design of detection and response strategies. In this thesis, the MITRE ATT&CK for ICS matrix is used both as a blueprint for constructing realistic attack scenarios and as the reference taxonomy for labeling technique identities in the collected data. For each scripted attack scenario, we follow the technique descriptions provided in the framework to drive how the attack is executed on the testbed, and we assign the corresponding technique labels to the gen-

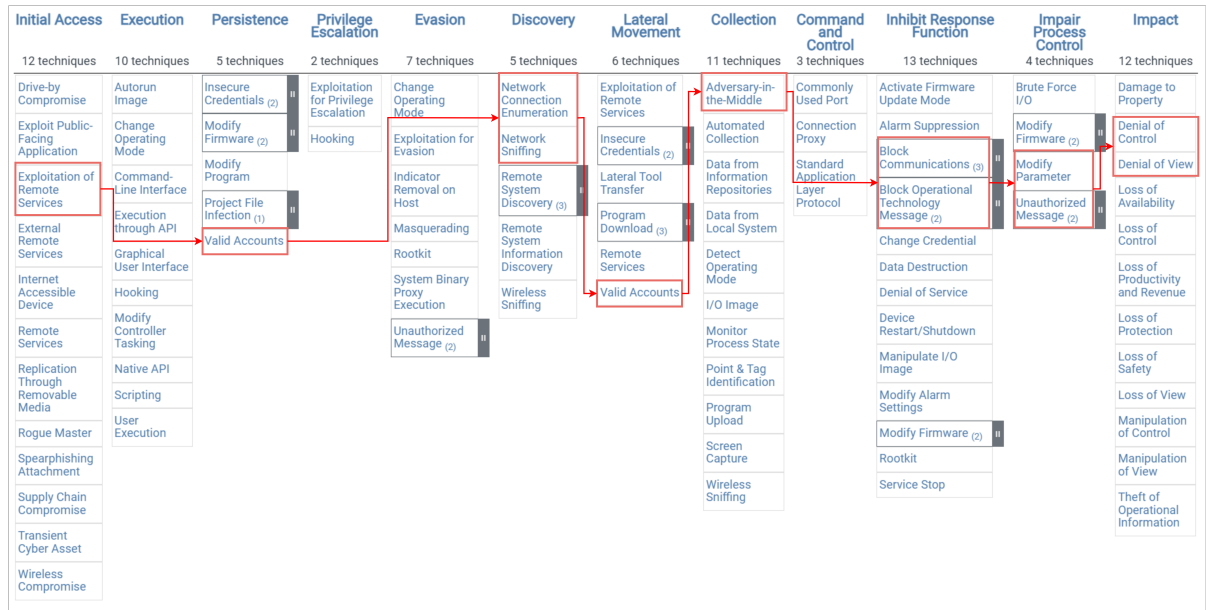


Figure 2.2: MITRE ATT&CK Framework for ICS & Example Kill Chain

erated alerts and telemetry. These labeled techniques then serve as the ground truth for training and evaluating our detection and classification models

Building upon the layered OT structure and attack progression described earlier, attacks in ICS environments often span multiple tightly coupled levels. A cross-level attack typically originates at higher-level components, such as supervisory systems at Level 2 (e.g., HMI/SCADA), and propagates downward through the control hierarchy. Once compromised, these supervisory systems can issue unauthorized yet syntactically valid commands to Level 1 controllers (PLCs), which in turn execute these commands and affect Level 0 physical devices, leading to unsafe or unintended physical process behaviors. These attacks exploit inherent trust relationships and control dependencies across OT layers. Because commands originate from trusted supervisory systems and follow legitimate communication channels, malicious actions may appear indistinguishable from normal operations when observed at an individual layer.

This multi-stage propagation creates a fundamental challenge for detection and response, as observable effects are distributed across supervisory logs, control commands, and physical process measurements. No single data source provides sufficient evidence to fully characterize the attack. Therefore, effective defense requires correlating information across OT levels and rea-

soning about their relationships, highlighting the need for cross-level analysis in both detection and response mechanisms.

2.3 AI in OT Security

Unlike IT environments, where a wide range of mature rule based security tools has been extensively developed, OT security solutions remain relatively limited due to the specific characteristics of industrial systems, including legacy devices, proprietary protocols, and strict operational constraints. In recent years, AI-based methods have attracted increasing interest for OT security, with a wide range of machine learning techniques [18] - from supervised and unsupervised learning to deep learning and reinforcement learning - being applied to automate key protection tasks, from anomaly detection to initiating response actions that preventing attacker escalation.

In term of anomaly detection, unsupervised learning (UL) is often preferred as it can model normal system behavior and detect deviations without requiring labeled attack data [19] - a resource that is typically scarce or incomplete in real-world ICS deployments [20]. The primary output of such UL models is an anomaly score or a binary decision (normal vs. anomalous), which is useful for spotting suspicious events but provides little semantic information about what type of attack may be occurring. To make these low-level anomalies more actionable, UL-based detectors are often complemented with feature-level analysis that highlights the most deviating signals or affected assets for each event, [21] so that the resulting alerts can be interpreted and investigated more easily by operators or SOC analysts. Once anomalies have been lifted to this textual alert level, the problem of understanding their security meaning can be formulated as a supervised classification task: mapping alert descriptions to attack techniques or tactics (e.g., following ATT&CK for ICS). Transformer models [22] are a natural fit for this setting, as they are designed for sequential data and can capture rich contextual relationships in text-based alerts; while they have been widely adopted in IT security for log and threat-intelligence analysis [23], their use for OT alert classification remains limited, which motivates further exploration in this direction.

In terms of automated response, reinforcement learning (RL) has emerged as a particularly attractive approach [24]. As illustrated in Figure 2.3, RL is a learning paradigm in which an agent interacts with an environment, observes the current state, selects actions, and receives feedback in the form of rewards; over time, the agent learns a policy that maximizes long-term return [25]. In OT settings, RL is especially appealing because response actions must avoid disruptive over-reaction [26]: by carefully defining the state representation and designing reward signals that penalize unsafe or overly aggressive interventions, RL agents can be guided to favour minimally intrusive mitigations that contain attacks while preserving process safety and availability

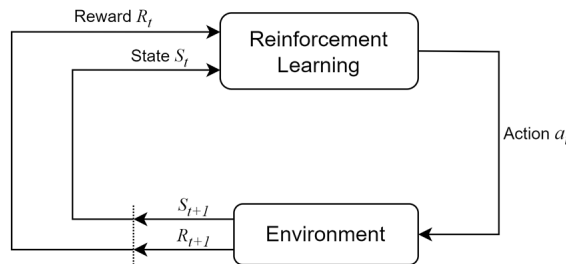


Figure 2.3: Reinforcement Learning

2.4 Related Works on AI-based Detection and Response in ICS/OT

Research on AI-based security for Industrial Control Systems (ICS) has largely developed along two parallel tracks: anomaly detection and automated response. In this section, we summarize representative OT-focused studies in Table 2.1, organized by their primary targets: detection-only, response-only, and integrated detection-and-response frameworks.

For each work, the table records its OT level coverage (whether it operates at Level 1, Level 2, or both) and the type of AI model used (e.g., supervised learning, unsupervised learning, reinforcement learning). The “Detection capability” columns indicate whether the method can only flag normal/abnormal behavior, or whether it also classifies alerts into specific attack types. The “Response capability” columns capture whether the approach explicitly reasons about security impact (reducing attack success) and about stability/safety of the underlying process,

i.e., whether mitigation actions are designed to avoid destabilizing or unsafe control behavior. The “Cross-level reasoning” column, in contrast, does not simply indicate that a work uses data from both Level 1 and Level 2, but whether it explicitly models correlations or joint reasoning across these OT layers. Finally, the table lists the testbed or dataset used for evaluation

Table 2.1: Related Works on AI-based Detection and Response in OT

Target	Paper	OT Level Coverage		AI Model	Detection capability		Response capability		Cross-Level Reasoning	Testbed environment
		L1	L2		Normal/ Abnormal	Attack Type	Security	Stability		
Detection	[7]	O	O	SL	O	-	-	-	O	SWaT
	[5]	O	-	UL	O	-	-	-	-	Ladder Logic Bombs for PLC simulation
	[6]	-	O	Semi-supervised Deep Learning	O	O	-	-	-	Public ICS dataset
Response	[11]	O	O	RL	-	-	O	O	-	ICS network simulation
	[12]	O	O	RL	-	-	O	-	O	Power grid voltage control CPS
	[9]	O	-	RL	-	-	O	O	-	GNS3 Network Emulation
Detection & Response	[13]	O	-	Rule Based + RL	O	-	O	O	-	Power grid / electric CPS
	[14]	O	-	Supervised CNN + RL	-	O	O	O	-	Industrial Internet of Things (IIoT)
	[15]	O	-	RL	O	-	O	O	-	IoT – Smart factory Testbed
	[10]	-	O	CNN+LSTM Deep RL	O	O	O	O	-	Public IoT dataset
	Ours	O	O	UL + Transformer + RL	O	O	O	O	O	ICS Simulation – Water Treatment

In the domain of intrusion detection, various machine learning (ML) paradigms have been applied to identify anomalies within Industrial Control Systems (ICS). For instance, studies such as [7] and [5] have leveraged supervised and unsupervised learning techniques to effectively classify anomalous system behaviors. However, a major limitation of these approaches is their restriction to binary classification—distinguishing only between normal and abnormal states—without identifying the specific attack type, thereby leaving defenders with ambiguous situational awareness regarding the actual security event. To address the challenge of scarce attack data for granular classification, [6] employed Deep Reinforcement Learning (DRL) to categorize specific attack techniques using public ICS datasets. Despite this advancement, a fundamental weakness persists: these datasets predominantly capture network-level data from supervisory

devices, lacking the physical process context required to analyze causal relationships between different operational level during a multi-stage intrusion. Consequently, models trained on such isolated data exhibit reduced robustness against sophisticated cross-level attacks. Furthermore, these studies only focus on detection, leaving the response process to human operators. This manual approach is not suitable for real-time ICS environments, as any delay in stopping an attack can lead to severe physical damage

To automate mitigation, several studies have applied Reinforcement Learning (RL) as an active defense mechanism [11] [12] [9]. The primary motivation of deploying RL-based response in OT environments is balancing security objectives—halting the attack—with the stability of physical system. An overreaction, such as abruptly shutting down a PLCs, can disrupt a continuous physical process and cause severe equipment damage comparable to the cyberattack itself. However, a major limitation of these RL-only frameworks is their strict reliance on independent, third-party detection tools. This decoupled approach treats detection alerts as black-box inputs, inherently restricting the agent’s observation space. Without direct access to raw data correlations, the agent’s capacity to execute precise, context-aware mitigation is significantly constrained.

Recognizing these inefficiencies, recent advancements [13] [14] [15] [10] have combined detection and response into a unified pipeline, yet they fundamentally lack robust cross-level reasoning. A comprehensive defense requires both modules to operate interactively across the system’s hierarchical levels. During detection, it is insufficient to merely collect data independently from the physical process (L1) and the supervisory network (L2); the system must analyze these streams in correlation to comprehend the causal semantics of multi-stage attacks. Similarly, the response mechanism must be capable of cross-level execution—coordinating sequential actions across L1 and L2 to halt an attack, while determining the most appropriate action for specific situations at each respective level.

To address the limitations of decoupled systems and the lack of cross-level reasoning, we propose an end-to-end AI agent framework for ICS/OT security. For detection, our pipeline

collects and correlates data from both the physical (L1) and network (L2) levels rather than analyzing them independently. This correlation provides the necessary context to identify cross-level attacks. Furthermore, the system maps these behaviors to the MITRE ATT&CK for ICS framework, detailing specific attack technique instead of just providing binary alerts. On the response side, we structure the RL agent's state and action spaces according to each operational control level to enable true cross-level reasoning. By incorporating physical variables and a well-designed reward policy, the agent learns exactly which actions are appropriate at which level. This structural design allows the agent to execute a coordinated sequence of actions across Level 1 and Level 2 to stop the attack effectively. Ultimately, the integration of correlated detection and sequential response creates an automated framework that can halt cyberattacks while maintaining the stability of physical processes.



Chapter 3

Problem Statement

This chapter introduces the defense problem in three parts: a notation table, which defines the key symbols and variables used throughout the chapter; a problem overview, which explains the cross-level attack kill chain and a problem statement, which formalizes the input, output, and optimization objective of the AI agent for detecting techniques and selecting response actions

3.1 Notations

To formulate the cross-level detection and response problem, we employ a consistent set of symbols, summarized in table 3.1. These notations are grouped into three categories: (1) Data sources, which describe multi-source OT-level measurements, (2) Alert events, which capture anomaly alerts and correlated alerts derived from those dataset, (3) Classification module, which represents predefined attack techniques and their mappings, and (4) Response module, which models the RL environment state, actions, rewards, and policy used by the AI Agent.

Table 3.1 lists each symbol together with its term and any specialized meaning used in later sections. In particular, $\mathcal{D}$ denotes the full collection of OT data sources, $d_{w,t}^l$ is a raw sample at time t from source w and level l , and $f_{w,t}^l$ is the corresponding feature extracted by the perception module. The alert variables $e_{w,t}^l$ and e_t^{corr} represent, respectively, per-source anomaly alerts and their correlated cross-source alert used as input to the AI agent. Within the detection module, $\mathcal{T}$ is the predefined set of attack techniques and $\hat{\tau}_t$ is the predicted attack technique.

For the response module, $N_{\mathcal{A}}$ denotes the number of predefined response actions and $\mathcal{A} = \{a_1, a_2, \dots, a_{N_{\mathcal{A}}}\}$ is the corresponding action set. The environment state $x_{m,t}^l$ summarizes critical OT variables at level l and OT variable index m , and together with alerts and detected techniques forms the RL state s_t observed by the AI agent. At each step, the agent selects an action $a_t \in \mathcal{A}$ according to a policy $\pi(a_t | s_t)$, receives an environment reward r_t , and updates

Category	Symbol	Term
Data source	$\mathcal{D} = \{D_w^l \mid w \in \{1, 2, 3\}, l \in \{1, 2\}\}$	Data source from multi source OT-level
	w	Source Type index: $w = 1$ – sensor, $w = 2$ – traffic, $w = 3$ – logs
	l	OT Level index: $l = 1$ – Level 1, $l = 2$ – Level 2
	$d_{w,t}^l$	Raw data sample from source w and level l in step t
	$f_{w,t}^l$	State feature extracted from raw data $d_{w,t}^l$ by Perception module
Alert event	$e_{w,t}^l$	Anomaly alert generated by the unsupervised learning model from source w and level l at step t
	e_t^{corr}	Correlated alert compiled from multiple raw alerts $e_{w,t}^l$ across different data sources and levels at step t
Classification Module	$N_{\mathcal{T}}$	Number of predefined techniques
	$\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{N_{\mathcal{T}}}\}$	Set of predefined attack techniques
	$\hat{\tau}_t \in \mathcal{T}$	Mapped attack technique for alert e_t^{corr}
Response Module	$N_{\mathcal{A}}$	Number of predefined response actions
	$\mathcal{A} = \{a_1, a_2, \dots, a_{N_{\mathcal{A}}}\}$	Set of predefined response actions
	$x_{m,t}^l, m \in \{1, 2, 3, 4, 5\}, l \in \{1, 2\}$	OT environment state m at level l and step t : $m = 1$ – device state, 2 – packet rate, 3 – tank overflow flag, 4 – unreachable device flag, 5 – isolated device flag
	m	OT environment state index: $m = 1$ – device state, 2 – packet rate, 3 – tank overflow flag, 4 – unreachable device flag, 5 – isolated device flag
	$s_t = (\hat{\tau}_t, \{e_{w,t}^l\}_{w,l}, \{x_{m,t}^l\}_{l,m})$	Cross-level RL state at step t , combining detected technique $\hat{\tau}_t$, multi-source UL alerts $e_{w,t}^l$, and OT environment state $x_{m,t}^l$
	$a_t \in \mathcal{A}$	Response action selected by RL agent for state s_t
	r_t	OT-environment reward for action a_t
	$\pi(a_t \mid s_t)$	Policy function mapping system state to an action
	$Q_{\theta}(s_t, a_t)$	Tabular Q-value for state-action pair

Table 3.1: Notation summary used in the proposed framework

the current OT state. These symbols provide the mathematical backbone for the following sections, where we describe how cross-level information is fused, how the AI agent reasons over techniques and states, and how the overall optimization objective is defined.

3.2 Problem Overview

In a typical ICS intrusion, the adversary first gains a foothold in the IT network (for example, through phishing or a compromised remote service) and then pivots toward the operational technology (OT) domain. Once inside OT, attacks rarely stay confined to a single layer: the

attacker often moves between Level 2 supervisory components (SCADA servers, HMIs) and Level 1 basic control devices (PLCs) while progressing along the ICS kill chain. Figure 3.1 illustrates this cross-level behavior within OT, where higher-level tactics such as discovery and lateral movement at Level 2 enable follow-up actions that manipulate PLC logic and physical processes at Level 1.

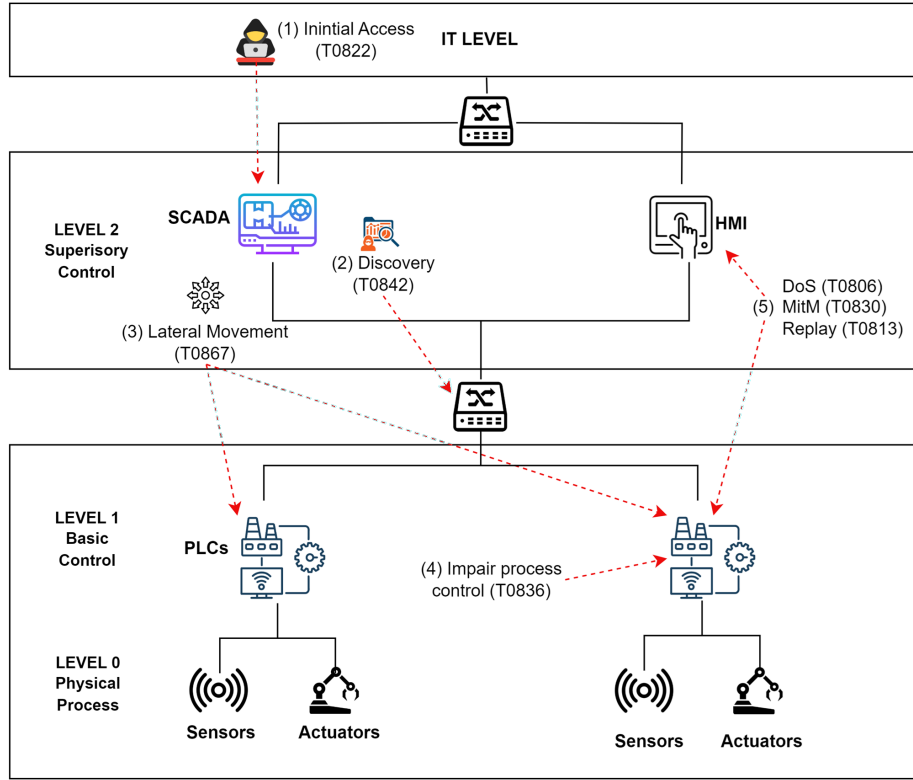


Figure 3.1: Example of Cross-Level Kill Chain in ICS-OT

To defend against this cross-level kill chain, we model a AI agent that continuously observes data and alerts from both Level 1 (Basic Control) and Level 2 (Supervisory Control). At each time step t , the OT environment produces multi-source data streams $d_{w,t}^l$ (for example, sensor values, network flows, or log events) from both levels, which are fed into the AI agent as shown in Figure 3.2 (AI Agent Architecture). Inside the agent, the Detection Module maps correlated alerts e_t^{corr} to an attack technique $\hat{\tau}_t$ over the predefined technique set T . This inferred technique, together with the current OT state, forms a cross-level state representation that encodes how the attacker's behavior spans Level 1 and Level 2.

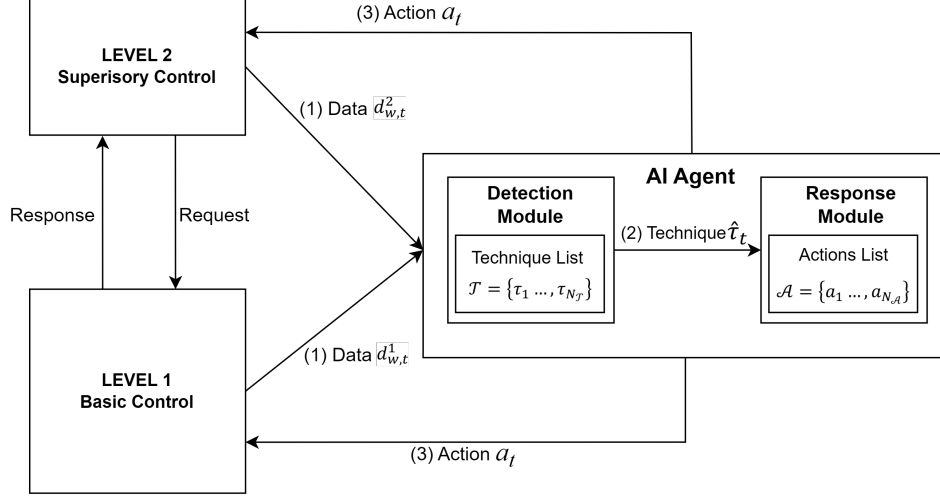


Figure 3.2: Problem Setting of Cross-Level AI Agent in OT

Based on this cross-level state, the Response Module selects a response action $a_t \in A = \{a_1, \dots, a_{N_A}\}$ such as throttling a request, isolating a PLC, or resetting a suspicious connection. The chosen action is sent back to the corresponding control level(s) as a mitigation command, closing the loop between detection and response. In this way, the AI agent does not react to isolated anomalies at a single layer but reasons over the entire multi-level kill chain: discovery or lateral movement at Level 2 can trigger preemptive actions at Level 1, and abnormal process control at Level 1 can be interpreted in the context of earlier IT or supervisory-level activity. The following problem statement formalizes this interaction as a sequential decision-making problem, specifying the input (multi-source data and alerts), output (technique predictions and actions), and the optimization objective for the AI agent.

3.3 Problem Statement

Industrial OT systems at Level 1 basic control and Level 2 supervisory control continuously generate heterogeneous data such as sensor readings, PLC traffic, SCADA/HMI logs, and other operational events. Within this environment, adversaries may execute multi-stage campaigns that traverse OT levels, while operators are limited to a finite set of predefined response actions that must satisfy safety and timing constraints. The central problem of this thesis is to build an AI agent that, based on multi-source observations from both OT levels and a predefined catalogue

of attack techniques and actions, can infer the most likely ongoing attack technique and choose an appropriate response so as to slow or prevent progression along the OT kill chain.

The problem addressed in this thesis can be formally defined as follows:

• **Input:**

- $\mathcal{D} = \{D_w^l \mid w \in \{1, 2, 3\}, l \in \{1, 2\}\}$: Multi-source OT-level data, where each source D_w^l represents data collected from source type w at OT level l .
- $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{N_T}\}$: Predefined attack technique set.
- $s_t = (\hat{\tau}_t, \{e_{w,t}^l\}_{w,l}, \{x_{m,t}^l\}_{l,m})$: RL state combining the detected attack technique $\hat{\tau}_t$, multi-source anomaly alerts $\{e_{w,t}^l\}_{w,l}$, and current OT environment states $\{x_{m,t}^l\}_{l,m}$
- $\mathcal{A} = \{a_1, a_2, \dots, a_{N_A}\}$: Predefined response action set.

• **Output:**

- $\hat{\tau}_t \in \mathcal{T}$: Predicted attack technique at decision step t .
- $a_t \in \mathcal{A}$: Selected response action at decision step t .

• **Objective:**

- Maximize the accuracy of detected attack techniques $\hat{\tau}_t$.
- Minimize the persistence and impact of the current attack technique $\hat{\tau}_t$ while avoiding OT system disruption caused by the response action.

• **Constraints:**

- The attack technique set $\mathcal{T}$ and response action set $\mathcal{A}$ are finite and predefined, ensuring safe implementation in the OT environment.

Chapter 4

Cross-Level Attack Detection and Response

This chapter presents the proposed AI agent architecture for cross-level attack detection and response in Industrial Control Systems. We first provide an overview of the integrated pipeline that combines multiple AI techniques to address the challenges of cross-level attacks. We then detail each component: unsupervised learning for anomaly detection, Transformer for attack technique classification, and reinforcement learning for automated response selection.

4.1 Solution Overview - Agent Architecture

As discussed in Chapter 3, many OT attacks exhibit cross-level characteristics, where the attack footprint appears simultaneously at multiple OT levels. To address this challenge, we propose a cross-level framework that jointly monitors multiple OT levels and heterogeneous data sources, including logs, network traffic, and sensor/process measurements. Instead of treating these sources independently, the system correlates anomaly evidence across levels and modalities into a unified cross-level alert representation. This design allows the framework to capture the relationships between multi-level effects, thereby improving attack visibility and enabling more context-aware reasoning in subsequent detection and response stages.

Building on this cross-level view, the first stage of the framework focuses on anomaly detection, which monitors all incoming OT data for abnormal patterns and raises alerts when deviations occur. OT environments are typically relatively stable, so normal operational behavior can be learned from historical data, whereas labeled attack traces are scarce and incomplete, making purely supervised detection hard to generalize. As a result, unsupervised learning becomes a natural fit for this setting. In our design, unsupervised models learn a baseline of normal behav-

ior from multi-source OT data (logs, network traffic, and process measurements) and generate alerts whenever new observations deviate from this learned baseline.

Once anomalous behavior has been detected, the next stage is to interpret these alerts in terms of concrete attack techniques. The correlated alerts produced by the first stage contain multi-source anomaly descriptions (e.g., textual alert messages and contextual metadata), so understanding the underlying attack requires both contextual and semantic reasoning. Transformer-based models are effective at learning semantic relationships in text and structured tokens, and their attention mechanisms help correlate heterogeneous pieces of alert evidence. Compared with large general-purpose language models, lightweight Transformers are more stable, more efficient, and easier to constrain, which leads to lower latency and more deterministic behavior—properties that are crucial in OT environments. Therefore, the framework employs a Transformer-based classifier to map cross-level anomaly alerts to MITRE ICS attack techniques.

After technique identification, the final stage of the framework focuses on response. Static rule-based response policies are difficult to maintain and cannot easily adapt to dynamic, multi-step attack situations. In ICS, response actions directly affect physical processes and operational stability, so overly aggressive mitigation can unintentionally disrupt the OT environment. As a result, response selection must explicitly balance attack mitigation with process safety and service continuity. Reinforcement learning offers a way to learn adaptive response policies through interaction with the environment, rather than hand-crafting all rules in advance. In the proposed design, the RL agent jointly observes the inferred attack techniques, correlated anomaly alerts, and current OT state, and chooses actions from a space that spans both Level 1 and Level 2 response options. This cross-level action space enables the agent to reason about coordinated mitigations across OT layers, leading to more context-aware and effective attack handling.

Figure 4.1 illustrates the overall system architecture, which consists of Four main modules operating in sequence:(1) a preprocessing module that transforms and organizes raw data into a model-ready format for subsequent machine learning modules (2) an anomaly detection module

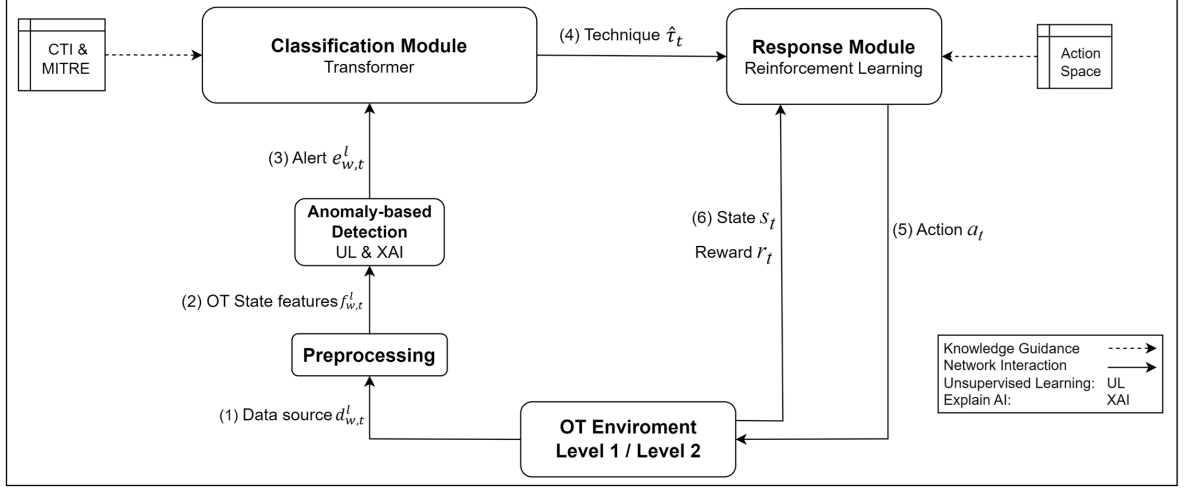


Figure 4.1: Solution Overview

based on unsupervised learning that identifies deviations from normal operational patterns, (3) a classification module that aggregates per-level alerts and classify those alert into attack techniques, and (4) a response module that selects coordinated mitigation actions across affected levels. The key innovation of this architecture lies in its cross-level information flow. Rather than treating Level 1 and Level 2 as independent security domains, the system maintains a unified view of the OT environment by correlating events across levels.

4.2 Attack Technique Detection

Figure 4.2 illustrates the proposed detection pipeline, which consists of multiple stages: raw data preprocessing, anomaly detection and alert generation, alert correlation, and attack technique classification. To capture cross-level attack behavior in OT environments, the proposed solution first collects heterogeneous data streams D_w^l from multiple sources. These include sensor data, network traffic, and system logs collected from both Level 1 and Level 2. The collected logs include controller logs, alarm messages, and operator actions.

Based on this integrated representation, an unsupervised learning model is trained using historical data that contains only normal operating behavior. The objective of this training phase is to learn the baseline characteristics of the water control process without relying on labeled attack samples. Then the trained model measures its deviation from the learned baseline, when

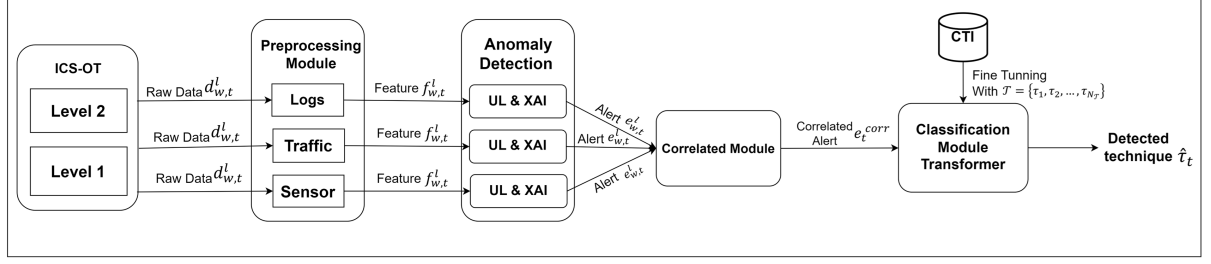


Figure 4.2: Technique Detection Solution

the resulting anomaly score exceeds a predefined threshold, the event is flagged as abnormal. To make these anomalies more informative, the framework then analyzes the per-feature deviations through anomaly score to determine which input variables contribute most to the anomaly score. This allows the system to localize the anomaly to specific sensors, network flow attributes, or log fields, and to construct enriched alerts $e_{w,t}^l$ that are forwarded to the next stage to classify the underlying attack technique.

After abnormal events are detected by the unsupervised learning, Correlation Module will aggregates temporally aligned multi-source anomaly alert $e_{w,t}^l$, including sensor data, network traffic, and system logs, into a single unified alert e_t^{corr} . To further enhance classification capability, this aggregation module also transforms raw anomaly alerts $e_{w,t}^l$ into semantically rich textual descriptions that better reflect their operational meaning. For example, an alert generated by UL model such as key: Max_tank_value, tag, value: 10 can be reformulated as "Malicious pattern: the maximum tank level at PLC 1 reached 10." This enriched textual representation provides more informative input for the transformer-based detection model.

The unified alert is then passed to a transformer-based text classifier that predicts the corresponding ATT&CK for ICS technique label. The correlated alert e_t^{corr} is first linearized into a single textual sequence that concatenates the multi-source anomaly descriptions in a fixed template. This sequence is tokenized and fed into a DistilBERT encoder with a sequence-classification head, which is fine-tuned on a private corpus of labeled alert data so that it learns to map alert narratives to specific techniques in the predefined set $\mathcal{T}$. At inference time, the model produces a probability distribution over $\mathcal{T}$ and outputs the most likely technique for the current alert, yielding a predicted label $\hat{\tau}_t$ that is forwarded to the reinforcement learning response agent.

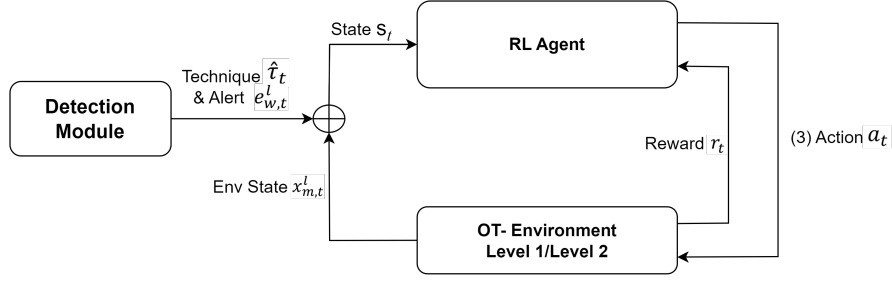


Figure 4.3: RL Module for Response

4.3 RL module for response

Figure 4.3 illustrates the structure of the reinforcement learning (RL)-based response module. In this module, the agent receives the current state s_t , which is constructed from two sources: the outputs of the detection pipeline and the current OT system status. Specifically, the detection-side information includes the predicted attack technique and anomaly observations derived from logs, sensor data, and network traffic, while the system-side information reflects the current operational state of the environment, such as PLC/HMI status and physical process conditions (e.g., tank level or flow state). By combining attack context with real-time system conditions, the state representation provides the agent with sufficient situational awareness to avoid unnecessary or overly disruptive responses. Based on this state, the agent selects an action a_t from the predefined response action space in order to stop or contain the attack while preserving system stability.

Table 4.1 summarizes the response action space used by the RL agent. To reflect the operational characteristics of ICS environments, the actions are organized along two dimensions: impact level and targeted OT level. In terms of impact, the actions are divided into four categories, namely low, medium, high, and critical, according to the degree of disruption they may introduce to system operation. Low-impact actions, such as alerting or terminating suspicious sessions, are generally less intrusive, whereas high- and critical-impact actions, such as modifying process parameters, restarting systems, or isolating devices, may directly affect process continuity and therefore must be applied more cautiously. In terms of targeted level, each action is associated with Level 1, Level 2, or both, depending on whether it affects field/control

operations, supervisory components, or cross-level communication. This organization makes the action space more consistent with ICS operational structure and allows the agent to learn response strategies that consider both security effectiveness and operational risk.

Table 4.1: Action table

Action	Description	Risk	Level
Alert	Send Alert to SOC	Low	L2/L1
Terminate SSH Session	Forcefully terminate active SSH sessions suspected to be used by attackers	Low	L2/L1
Remove Attacker File	Remove malicious files that attacker agents deployed during lateral movement stage	Low	L2
Rate Limit	Limit traffic in/out devices to reduce excessive requests or flooding	Medium	L2/L1
ARP Table block	Preventing Device to update a new ARP Entry	Medium	L2/L1
Disable Write Request	Disable write operations to prevent attackers from modifying PLC registers or control parameters.	Medium	L1
Disable User Account	Temporarily disable compromised or suspicious user accounts.	Medium	L2
Change Credential	Reset password	Medium	L2/L1
Block Read request	Block unauthorized or suspicious read requests to PLC registers	High	L1
Modify Physical Parameter	Restore/Adjust manipulated physical tank/bottle level values to normal values	High	L1
Terminate HMI Session	Terminate suspicious or unauthorized HMI user sessions	High	L2
Restore System State	Restart devices	Critical	L2
Isolate device	Disconnect a compromised device from the network	Critical	L2/L1

We adopt tabular Q-learning, as shown in Algorithm 4.1, to implement the RL-based response policy. This choice is appropriate for our setting because the response action space is relatively small and discrete, allowing efficient updates without requiring deep neural networks or large computational resources. In addition, tabular Q-learning provides fast learning and straightforward interpretability, which are desirable properties for adaptive response in ICS environments.

After observing the current state, the agent selects a response action and evaluates its effect through the next state. The reward is designed to balance two objectives: stopping the attack and preserving system stability. A positive reward is assigned when the action restores detection-related indicators to a normal state, while a penalty is given if abnormal behavior persists. Additional penalties are introduced when the action causes unnecessary disruption to OT operations, such as disconnecting critical devices or affecting physical processes. To be more

Algorithm 4.1 RL-based Response Agent for OT Attack Mitigation

Inputs / constants:
 $\mathcal{T}$: Attack technique set, $\mathcal{A}$: Response Action Set

 $\mathcal{S}$: State Space (Technique $\hat{\tau}_t$ + UL alerts $\{e_{w,t}^l\}_{w,l}$ + OT states $\{x_{m,t}^l\}_{l,m}$)

 γ (discount), α (learning rate), $\epsilon_0, \epsilon_{min}$ (initial & minimum exploration), ρ (exploration decay)

 $T_{max} = 10$ (Maximum steps per episode)

```

1: Initialize  $Q_\theta(s_t, a_t)$ 
2: for episode = 1, 2, ... do
3:    $\epsilon \leftarrow \epsilon_0$ ,  $\hat{\tau}_t \sim \text{Uniform}(\mathcal{T})$  ▷ random attack technique
4:    $s_t \leftarrow \text{MAPPER}(\hat{\tau}_t, \{e_{w,t}^l\}_{w,l}, \{x_{m,t}^l\}_{l,m})$ 
5:    $N_t \leftarrow \text{COUNT\_ANOMALY}(s_t)$  ▷ Number of abnormal states
6:    $stable\_count \leftarrow 0$  ▷ Episode termination condition
7:   while  $stable\_count < 2$  and  $t < T_{max}$  do
8:     /*  $\epsilon$ -greedy action selection */
9:     With prob.  $\epsilon$ :  $a_t \leftarrow \text{Uniform}(\mathcal{A})$ 
10:    Otherwise:  $a_t \leftarrow \arg \max_{a'} Q_\theta(s_t, a')$ 
11:    Execute response action  $a_t$ 
12:    Observe:  $s_{t+1} \leftarrow \text{MAPPER}(\hat{\tau}_t, \{e_{w,t+1}^l\}_{w,l}, \{x_{m,t+1}^l\}_{l,m})$ 
13:     $N_{t+1} \leftarrow \text{COUNT\_ANOMALY}(s_{t+1})$ 
14:    /* Reward Calculation */
15:     $r_t \leftarrow -0.05$  ▷ step penalty to discourage long episodes
16:     $r_t \leftarrow r_t + 1.8(N_t - N_{t+1})$  ▷ reward when anomaly count decreases
17:     $r_t \leftarrow r_t - 0.30N_{t+1}$  ▷ penalize remaining anomalies
18:    if  $N_t > 0 \wedge N_{t+1} = 0$  then
19:       $r_t \leftarrow r_t + 3.0$  ▷ Big reward if attack is fully mitigated
20:     $stable\_count \leftarrow stable\_count + 1$  if  $N_{t+1} = 0$ , else 0
21:    /* Update Q-Network */
22:     $y_t \leftarrow r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a')$ 
23:    Update  $\theta$  using:  $(y_t - Q_\theta(s_t, a_t))^2$ 
24:     $s_t \leftarrow s_{t+1}$ 
25:     $\epsilon \leftarrow \max(\epsilon\rho, \epsilon_{min})$ 

```

detail, action that cause disruption in Level 1 will achieve higher penalty compared with Level 2. Over time, this reward design enables the agent to learn a mitigation policy that stops attacks effectively while avoiding overreaction. After the action is executed, the environment transitions to the next state s_{t+1} , which is updated based on both the new detection outputs and the updated operational status of the system. The episode will over if the agent successfully mitigated the attack, or reach the max step.

The Q-value update follows the standard Bellman formulation. For each transition (s_t, a_t, r_t, s_{t+1}) , the target value is computed as the immediate reward plus the discounted maximum future Q-value of the next state, unless a terminal state is reached. The corresponding Q-table entry is then updated accordingly. After each step, the exploration rate ϵ is decayed to gradually shift the

learning process from exploration to exploitation. This procedure is repeated across episodes, allowing the agent to progressively learn a response policy that mitigates attacks while minimizing unnecessary disruption to ICS operations.



Chapter 5

Implementation

This chapter describes the comprehensive implementation of our AI-based cross-level attack detection and response framework within an ICS/OT environment. We first outline the open-source tools and libraries utilized to construct the operational environment and develop the intelligent models (Section 5.1). Following this, we detail the testbed setup, which encompasses the configuration of a multi-level simulated water treatment facility, the execution of multi-stage attack scenarios, and the data collection process (Section 5.2.1). Finally, we present the technical implementation of our proposed defense solution, detailing how the machine learning (ML) and reinforcement learning (RL) models are instantiated and integrated to secure the system (Section 5.2.2).

5.1 Open Source Tools and Libraries

Table 5.1 summarizes the key open-source components utilized in our system, systematically categorized into testbed emulation, threat execution, network analysis, and AI development. At the experimental core is ICSSIM [27], deployed via Docker containers [28] to replicate a multi-level ICS architecture and generate synchronized multi-source data—including sensor readings, system logs, and network traffic—essential for training our AI models. To simulate sophisticated adversaries, we leverage the MITRE ATT&CK for ICS framework [3] and MITRE CALDERA [29] to structure and launch advanced, coordinated attacks against this testbed. The resulting network flows and system behaviors are then comprehensively captured and analyzed using ICSflow [30], Wireshark [31], rsyslog [32], and tcpdump [33].”

Finally, the analytical and artificial intelligence components are entirely orchestrated in Python. For feature engineering, and core machine learning computations, we rely on NumPy [40], Scikit-learn [34], and Gensim [41]. The Reinforcement Learning (RL) agent, which drives the

Category	Name	Functionality
Tools	MITRE Caldera for OT [29]	Attack generation
	ICSSIM [27]	ICS environment deployment
	Docker [28]	Containerize testbed
	ICSFlow [30]	Network traffic flow extraction
	Wireshark [31]	Collect and analyze network data
	Rsyslog [32], Tcpdump [33]	Syslog and network traffic recording
Python Libraries	Scikit-learn [34]	Machine learning
	PyTorch [35], RLlib [36]	Implementing RL algorithms
	Matplotlib [37], Pandas [38], Seaborn [39]	Visualization and data analysis

Table 5.1: Tools and Python libraries used in our implementation

automated response mechanism, is implemented using the robust PyTorch [35] framework combined with RLlib [36]. To process the generated datasets and visualize the evaluation metrics, we utilize industry-standard data manipulation and visualization libraries, including Pandas [38], Matplotlib [37], and Seaborn [39].

5.2 Implementation Details

Figure 5.1 illustrates the overall implementation, which couples a Docker-based ICS testbed with an AI-driven monitoring and response agent. The testbed emulates a water treatment process spanning Level 2 supervisory control (HMI and SCADA services) and Level 1 physical control (PLCs, sensors, and actuators), connected via an IP network and including an attacker node that generates cross-level attack scenarios. This testbed generates realistic multi-source data in the form of process sensor readings, network traffic, and system logs. On the right, the AI agent processes these data through a modular pipeline that includes data preprocessing, unsupervised anomaly detection, Transformer-based classification of attack techniques, and an RL-based response module that selects mitigation actions based on the detected technique and current system state.

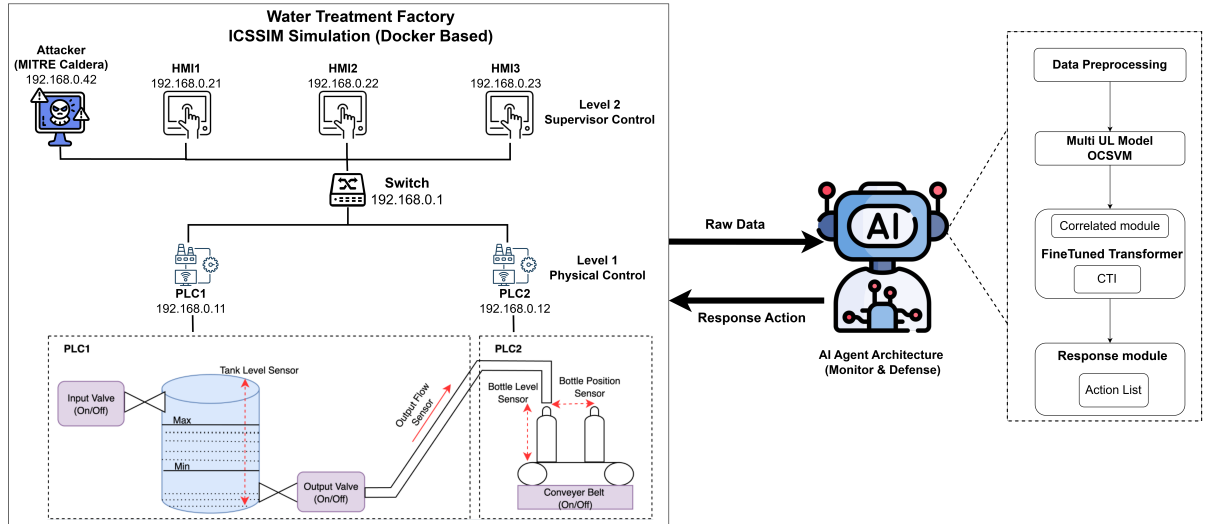


Figure 5.1: Implementation Overview

5.2.1 Testbed

The proposed framework is deployed on a host machine running Ubuntu 20.04, equipped with an Intel Core i7-14700F CPU, an NVIDIA GeForce RTX 4070 GPU, and 32GB of RAM. The testbed is designed to simulate a real-world Cyber-Physical System (CPS) for a water treatment facility. As illustrated in Figure 5.1, The testbed is built upon the ICSSIM framework using a Docker-based containerized environment. It consists of six primary containers categorized by their operational hierarchy:

- Supervisory Control Level (L2): Three HMIs run with `HMI1.py`, `HMI2.py`, `HMI3.py`. Specifically, HMI1 serves as a monitoring interface, while HMI2 and HMI3 are designated for active system control.
- Physical Control Level (L1): PLC1 is responsible for monitoring tank values and regulating water output, whereas PLC2 controls the conveyor belt and bottle fluid levels, both container run `PLC1.py` and `PLC2.py` accordingly. A dedicated Python-based simulation module `pys.py` simulates the raw physical processes, executing the corresponding sensor and actuator dynamics in real-time.

To generate realistic attack data, an adversary machine is deployed on the same local network utilizing the MITRE CALDERA framework. For this research, we assume the attacker has

ID	Name	Description
T0840	Network Connection Enumeration	Capture network traffic to observe communications and extract IP addresses, host names, and device properties.
T1110	Brute-force SSH	Repeatedly attempt passwords to gain unauthorized SSH access.
T0864	Transient Cyber Access	Login to the system using an unauthorized session.
T0892	Change Credential	Modify authentication data (e.g., passwords).
T0886	Exploit SSH Remote Service Lateral Movement	Exploit SSH services to deploy attacker agents across multiple devices.
T0830	Man in the Middle	Intercept and alter communications between HMI and PLC.
T0861	Point and Tag Identification	Identify control tags used in sensors and actuators.
T0836	Modify Parameter	Change control parameters of PLC, such as tank min/max values.
T0855	Unauthorized Command Message	Send unauthorized control commands from HMI to PLC.
T0803	Block Command Message	Block legitimate control commands between HMI and PLC.
T0814	Denial of Service	Flood PLC devices with high-frequency read commands to disrupt operation.

Table 5.2: Attack techniques used in the ICS testbed based on MITRE ATT&CK for ICS

already bypassed the enterprise IT perimeter and established an initial foothold within the OT network.

We designed a comprehensive, multi-stage attack chain encompassing 11 distinct MITRE ATT&CK for ICS techniques. This kill chain progresses from initial network discovery to ultimate physical impact. Crucially, to evaluate the framework's cross-level reasoning capabilities, the dataset includes 6 single-level techniques and 5 advanced cross-level techniques (T0886, T0830, T0855, T0836, T0803), where the attack patterns span both the supervisory and physical level. Table 5.2 summarizes the executed attack sequence.

- **T0840 - Network Connection Enumeration:** The attacker uses Nmap to scan the OT network. The goal is to gather all information of OT network such as hostname, open ports, operating systems, and exposed services,...
- **T1110 - Brute-Force SSH:** Using the information gathered from scanning, the adversary targets specific hosts (HMI, PLC) and performs SSH brute-force attempts to guess valid credentials. The objective is to exploit weak authentication mechanisms and extract valid administrative credentials for the HMIs and PLCs.
- **T0864 - Transient Cyber Access:** After obtaining valid credentials, the adversary logs

into multiple devices as a legitimate user. These transient logins allow them to explore configurations, pivot laterally, and prepare further malicious actions.

- **T0892 - Change Credential:** To maintain persistence and evade defensive interventions, the attacker alters the system passwords on the compromised devices. This action locks out legitimate plant operators, securing the attacker's unauthorized control over the environment.
- **T0886 - Remote Services (Cross-level):** Leveraging SSH services, the adversary deploys additional agents to other hosts in the OT network. This lateral movement expands their foothold and enables coordinated attacks across many devices.
- **T0830 - Man in the Middle (Cross-level):** By executing an ARP spoofing attack between PLC1 and HMI1, the adversary intercepts Modbus/TCP traffic. They inject forged control messages at the network level (L2) while simultaneously altering true L1 sensor readings before they reach the operator.
- **T0861 - Point and Tag Identification:** The adversary queries the OT environment (for example, exploiting access to a SQLite tag database) to enumerate process tags and their meanings. This reveals which variables correspond to physical assets, enabling more precise and impactful manipulation later
- **T0836 - Modify Parameter (Cross-level):** Using tools such as `modbus_cli`, the attacker issues write requests that change critical process parameters (e.g., tank level thresholds or conveyor speed limits). These modifications could potentially cause physical disruptions like water tank overflow,...
- **T0855 - Unauthorized Command Message (Cross-level):** Leveraging the compromised control interface (HMI2), the attacker issues unauthorized operative commands to the PLCs. This intentionally forces L1 physical equipment, such as water valves and conveyor belts, to deviate dangerously from their safe operational states.
- **T0803 - Block Command Message (Cross-level):** The attacker blocks command and write messages from monitoring and control HMIs to the PLCs. Operators can no longer

adjust process values or reliably observe system state, losing the ability to correct previous malicious changes.

- **T0814 - Denial of Service:** In the final phase, the attacker spawns hundreds of synthetic HMI instances using automated scripts. These instances aggressively flood the PLCs with concurrent Modbus requests, exhausting device resources and causing a complete halt and become unresponsive.

5.2.2 Implementation of AI Detection and Response Pipeline

This section describes the implementation of the four-stage AI pipeline as illustrate in Figure 5.1: data collection and preprocessing, unsupervised anomaly detection, transformer-based attack classification, and reinforcement learning-based response. Each stage is implemented as a modular component with well-defined interfaces, enabling independent development and integration into the broader system. The AI pipeline is executed on a local workstation with the following hardware and software configuration:

- **CPU:** Intel Xeon w5-3435X (16 cores, 32 threads)
- **RAM:** 128 GB
- **GPU:** $2 \times NVIDIA RTX 4090 (24GB VRAM each)$
- **Operating System:** Ubuntu 22.04.4 LTS (Kernel 6.8)
- **CUDA Version:** 12.4 (Driver 550.144.03)

Raw multi-source data is continuously collected from three distinct OT data streams: sensor readings from Level 1 (tank levels, flow rates, valve states,...), system logs from both Level 1 and Level 2 (PLC logs, HMI logs), and network traffic (Modbus/TCP flows between HMIs and PLCs). All data is captured in both normal operations and attack scenarios. Data is stored in source-specific formats: network traffic as PCAP files (processed via ICSFlow to extract flow-level features), system logs as syslog streams, and sensor data as time-series CSV files with numeric process variables. This raw data serves as input to source-specific preprocessing pipelines.

Sensor data is collected as multivariate time-series measurements from physical process variables such as tank levels, flow rates, and valve states. Since these attributes are already numeric, preprocessing focuses on converting all values to a consistent numeric format, replacing missing values (NaN) with zero, and removing non-informative metadata such as absolute timestamps. The remaining process variables are standardized using zero-mean, unit-variance scaling so that all sensor features are represented on a comparable scale for subsequent analysis.

Raw network traffic is aggregated into flows using ICSFlow. Each flow contains approximately 60 statistical features (packet count, byte volume, inter-arrival times, protocol-specific fields). Manual feature selection reduces this to the 20 most informative features while excluding contamination attributes (source IP, destination IP, port numbers, which would limit model generalization). Preprocessing includes normalization and handling of missing values.

System logs are tokenized and converted to numeric vectors using Word2Vec embedding. TF-IDF weighting identifies the most important terms within each log entry. Contamination features (IP addresses, port numbers, device names, timestamp metadata) are removed to prevent the anomaly detector from memorizing spurious correlations. Logs are then represented as feature vectors suitable for OCSVM training.

For each data source, a separate One-Class SVM (OCSVM) model is trained on the normal-only training data to capture the statistical boundary of benign behavior. At test time, data points that fall outside this boundary are flagged as anomalies. Feature extraction is then applied after an anomaly is detected, feature-level importance scores are computed using z-score deviation from the normal distribution mean and standard deviation. For each test sample, the z-score of every feature is calculated, features are ranked by their z-score magnitude, and the top-3 features with the highest deviation are selected as primary contributors. Features below a predefined threshold are filtered to reduce noise. For each detected anomaly, an alert is generated containing Timestamp and duration of the anomaly, Anomaly severity score (distance from the normal boundary) and Top-3 contributing features with their abnormal values.

```

"timestamp": "2026-04-06 16:26:05",
"logs_alert": {
  "logs": [
    "[HMI2] [OK] Set FULL level of bottle → 5.00",
    "[HMI2] [AUDIT] Operator changed bottle_level_max to 5.00"
  ]
},
"traffic_alert": null,
"sensor_alert": {
  "messages": [
    "Malicious: PLC2_bottle_level_max(11) is 5.0, contribution is 3200000.0000000005.",
    "Malicious: PLC1_tank_level_max(4) is 10.0, contribution is 3000000.0.",
    "Malicious: PLC1_tank_level_min(3) is 1.0, contribution is 2000000.0."
  ]
}

```

Figure 5.2: Example of Correlated Alert

A correlation module aggregates alerts from multiple sources (sensor, network, log) into a single, structured cross-level alert by applying temporal windowing, deduplication, and natural-language enrichment. Incoming per-source alerts are buffered and grouped when their timestamps fall within a short configurable window (prototype: 1 second), and if multiple identical alerts originate from the same source within this window, only the first alert is kept to reduce noise from repeated triggers. The module then converts numeric features and ranked contributing features into short human-readable phrases—for example, a network feature `sPackets` with value v is rendered as “Number of packets sent from source to destination in this flow is v ” — and concatenates them into a compact alert narrative. The correlated alert, as shown in Figure 5.2 is a semantically rich cross-level alert document that preserves temporal order and causal cues across devices such as HMI and PLC in both Level.

A Transformer is applied to classify those alerts. Each alert is converted into a single linearized text sequence by extracting evidence from sensor, log, and traffic fields. The resulting sequence uses lightweight source markers, namely [SENSOR], [LOG], and [TRAFFIC], and messages within each source are normalized, deduplicated, sorted, and concatenated with separator symbols. Environment-specific values such as IP addresses, ports, numerical values, contribution scores, and physical tag names are replaced during preprocessing with generic placeholders such as <IP>, <PORT>, <NUM>, <SCORE>, and <SENSOR_TAG>,... reducing dependence on plant-specific identifiers.

The classifier is implemented by fine-tuning distilbert-base-uncased with a sequence-classification head using cross-entropy loss. The linearized alert text is tokenized and padded or truncated to 256 tokens. The model produces one logit score per technique, and the final prediction is selected using argmax. The task is formulated as single-label multi-class classification. For evaluation, alerts from each technique are first ordered by timestamp. The earlier 70% of alerts are used for training, while the later 30% are kept only for testing. Evaluation uses accuracy, macro precision, macro recall, macro F1-score, confusion matrices, and inference time. The predicted technique identifier is then attached to the alert and passed to the reinforcement-learning response module as part of its input state.

The response module is implemented using a Deep Q-Learning agent operating on a discrete action space across both Level 1 and Level 2 of the ICS environment. At each decision step, the agent observes a state vector composed of two main components: (1) detection outputs, including the detected attack technique identifier and anomaly status derived from monitored data sources, and (2) physical process indicators, including sensor abnormality flags, PLC state, and HMI state. The action space consists of predefined mitigation actions at two levels as illustrated in Table 4.1, and the agent selects one action at each step.

After executing an action, the environment updates the system state and returns a scalar reward. A positive reward is assigned when the abnormal condition is reduced or the attack is mitigated, while a negative reward is assigned when the action fails to reduce abnormality or introduces additional disruption. Training is performed using an epsilon-greedy policy, where the agent selects a random action with probability ϵ or the action with the highest Q-value otherwise. Transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$ are stored in a replay buffer and sampled in mini-batches to update the Q-network. A target network is used and periodically synchronized with the online network. Each episode represents a single attack mitigation process and terminates when the attack is mitigated (all system states return to normal) or the maximum number of steps (10) is reached.

Chapter 6

Evaluation and Results

This chapter presents the experimental results. We first describe the evaluation setup, including the ICS testbed, the three agent architectures under comparison, and the metrics used to quantify both detection quality and response effectiveness (Section 6.1). Section 6.2 then answers Research Question (RQ) 1 by assessing whether the cross-level agent can detect and mitigate attacks more effectively than single-level agents, using macro-averaged technique-classification metrics and attack-blocking success rates. Section 6.3 addresses RQ2 by examining the performance of the detection module from two perspectives: the classification module, where LLM-based and transformer-based approaches are compared in terms of F1-score, precision, recall, and inference time; and the anomaly detection module, where multi-source inputs are compared with single-source settings. Finally, Section 6.4 investigates RQ3, analyzing how different state definitions—particularly the inclusion of carefully designed physical process variables—affect the agent’s response behavior and its ability to avoid over-reaction while still stopping attacks.

6.1 Experimental Setup & Performance Metrics

The evaluations are conducted on a two-machine setup that separates the industrial process emulation from the AI control plane. The first machine runs Ubuntu 20.04 and hosts the ICS testbed, including the simulated physical process, PLC logic, network services, and attack scripts responsible for generating multi-source telemetry (sensor streams, network traffic, and logs). The second machine runs Ubuntu 22.04 and hosts the AI components: the transformer-based detection pipeline and the reinforcement learning (RL) response agent. We compare three agent architectures that differ in how much of the OT stack they observe and protect. The cross-level agent receives full multi-source input from both Level 1 and Level 2 and can issue coordinated actions on both layers. The Single-L1 agent observes only Level 1 process/control signals and is restricted to actions that affect Level 1, while the Single-L2 agent is limited to Level 2 su-

pervisory/network signals and actions. This design allows us to isolate the effect of cross-level visibility on both detection and response.

For the detection module, we use `distilbert-base-uncased` as the main classifier, as it offers a good trade-off between representational capacity and inference cost. A maximum sequence length of 256 tokens is sufficient to cover the linearized alert narratives without excessive truncation, while keeping memory usage low. We train with a batch size of 16 for both training and evaluation, which fits comfortably on a single GPU and stabilizes gradient estimates, and we run 5 epochs to reach convergence without overfitting the relatively modest alert corpus. Detection performance is measured using macro-averaged F1-score, precision, and recall to capture per-technique behavior, complemented by confusion matrices and average inference latency per alert to assess suitability for online use. The response agent is implemented using Deep Q-Learning (DQN). We set the learning rate to 10^{-3} and the discount factor $\gamma = 0.99$ to favor long-term mitigation effects while still allowing learning to progress at a reasonable pace. The exploration parameter ϵ is annealed from 1.0 to 0.05 with a decay factor of 0.998, encouraging broad exploration early in training and more greedy behavior later. The agent is trained with a replay batch size of 128 and a target-network update every 200 steps over 3000 attack episodes, which empirically provides stable convergence. RL performance is evaluated using the average attack-blocking success rate, per-technique and per-stage success rates along the ICS kill chain, the cumulative reward over episodes, and snapshots of the learned Q-values and action preferences for each technique to interpret the resulting policies.

6.2 Cross Level vs Single Level

Figure 6.1 compares the technique-classification performance of the cross-level transformer model against two single-level baselines on the held-out test set. The cross-level configuration achieves the highest macro-averaged F1-score of 0.97, compared to approximately 0.87 for the Level 2-only model and 0.81 for the Level 1-only model, and the same pattern holds for macro precision and recall. This gain can be attributed to the richer context available when alerts from multiple OT levels and data sources are combined into a single input sequence: the cross-level

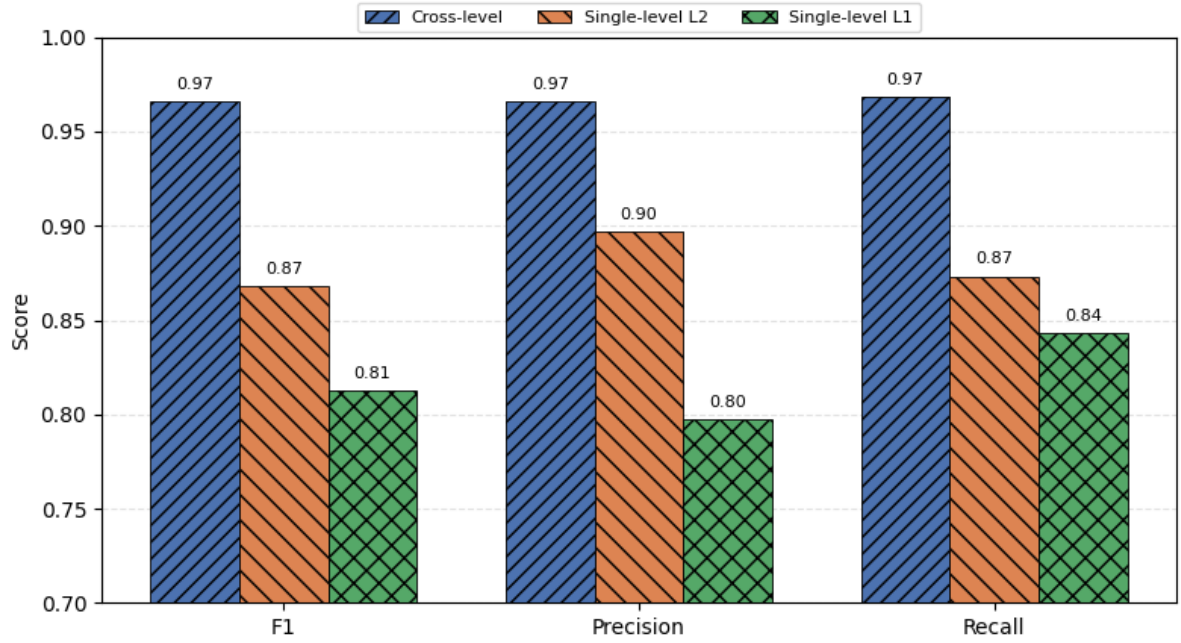


Figure 6.1: Overall Detection Performance Across Agents

model sees both supervisory/network symptoms and process-level indicators, making it easier to disambiguate techniques that would look similar if only one layer were observed. By contrast, the single-level models operate on a partial view of the attack behavior and therefore miss some cross-level cues, which leads to more misclassifications. Overall, these results indicate that cross-level detection is more effective than single-level detection for transformer-based ATT&CK for ICS technique classification in our testbed.

Figure 6.2 provides a more detailed view of the technique-classification results by showing the confusion matrices for the cross-level, Single L1, and Single L2 models. The cross-level classifier achieves almost perfect per-class accuracy, with only a small fraction of T0886 (lateral movement) episodes misclassified as T1110 (brute-force SSH) and a few T0814 (DoS) samples confused with T0842. In contrast, the Single L1 and Single L2 models exhibit much larger class-specific errors, particularly on techniques that share similar symptoms when observed from only one level. For example, the Single L1 model often mislabels T0886 as T1110, and struggles to distinguish T0836 (modify parameter) from T0855 (unauthorized command message) because both primarily manifest as abnormal process values at Level 1, even though their network and log

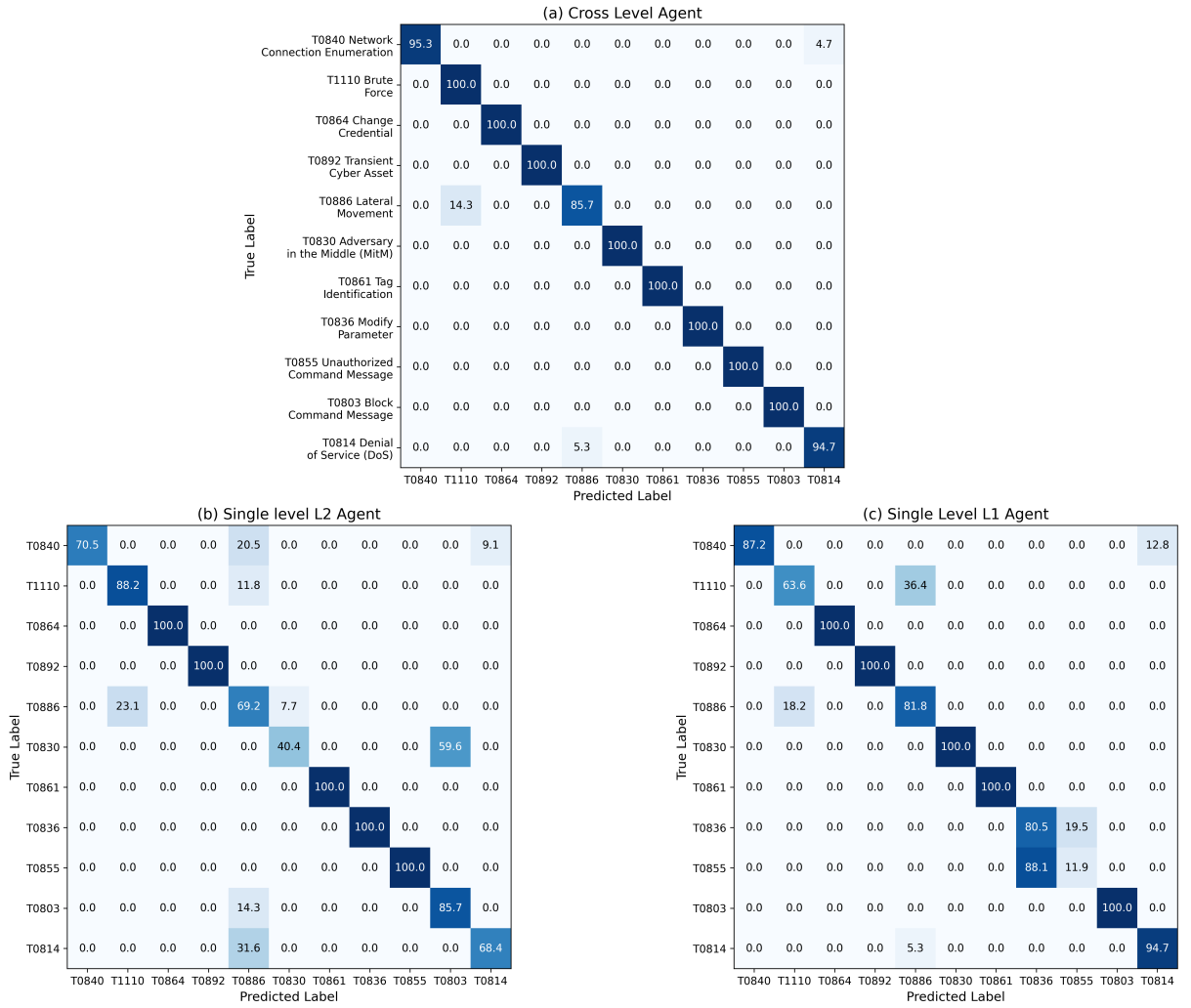


Figure 6.2: Confusion matrices of three agent configurations

patterns at Level 2 are different. The Single L2 model shows the opposite weakness: it confuses techniques such as T0840 (network connection enumeration) and T0814 (DoS), or T0803 (block command message) and T0830 (man-in-the-middle), which all generate similar traffic patterns at Level 2 but differ in how they impact the physical process. These results illustrate that good overall macro scores can hide fragility on specific cross-level techniques, and that combining Level 1 and Level 2 evidence is essential to robustly disambiguate techniques whose behaviors span multiple OT layers.

Figure 6.3 tracks the global success rate of the three response agents over 3000 attack sessions. For each episode, we record a binary outcome: an episode is counted as a success if the agent restores the OT system to a normal state before reaching the maximum step budget, and as a failure otherwise. To obtain a smooth curve, each point on the plot shows the average suc-

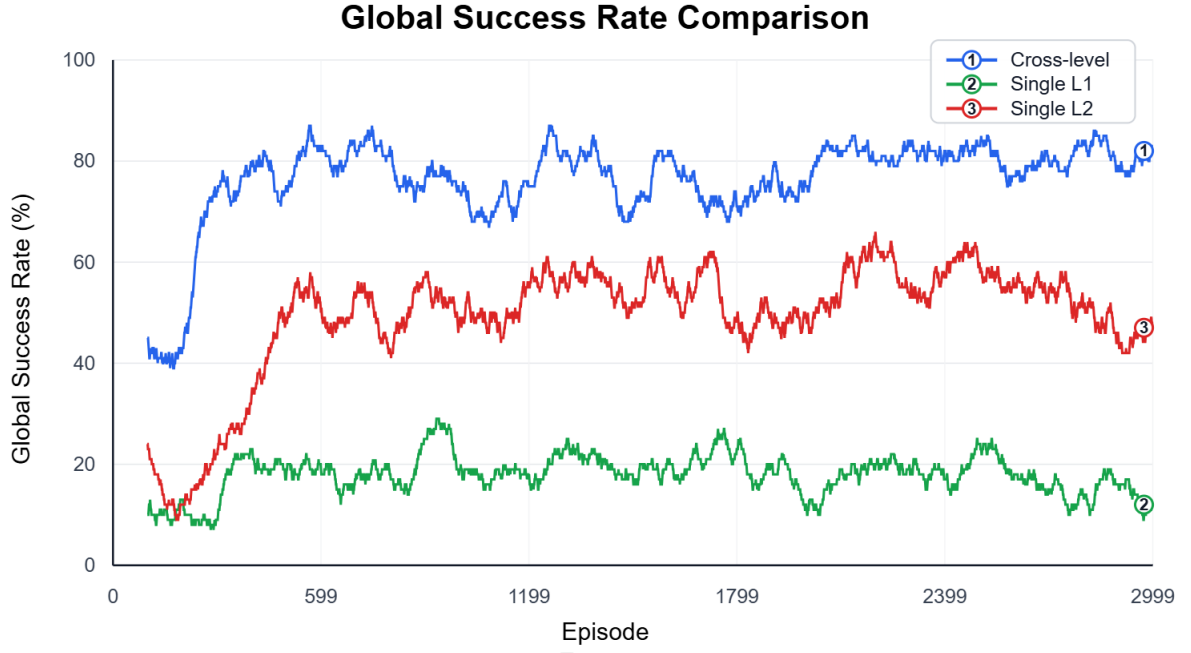


Figure 6.3: Global Success rate over Training Episodes

cess rate over a sliding window of 100 episodes (for example, if 65 out of the last 100 episodes are successful, the plotted value is 65%). The x-axis corresponds to the episode index from 0 to 2999, and the y-axis reports the success rate in percent for the Cross-level, Single-L1, and Single-L2 agents. The Cross-level agent achieves the highest and most stable success rate, converging to values around 80% after the initial learning phase, which indicates that observing and controlling both Level 1 and Level 2 jointly enables it to stop cross-level attacks more reliably. The Single-L2 agent attains an intermediate success rate of roughly 50–60%, outperforming the Single-L1 agent because many attack scenarios manifest more clearly at the supervisory/network layer, but still lagging behind the Cross-level agent due to its lack of detailed process visibility. The Single-L1 agent performs poorest, with a success rate of only about 15–25%, suggesting that process-level information alone is insufficient to select effective responses for most scenarios. Overall, the figure shows that the Cross-level RL agent not only improves its performance substantially over time but also maintains a high steady-state success rate, supporting the feasibility of the proposed cross-level approach for mitigating attacks in multi-layer ICS environments.

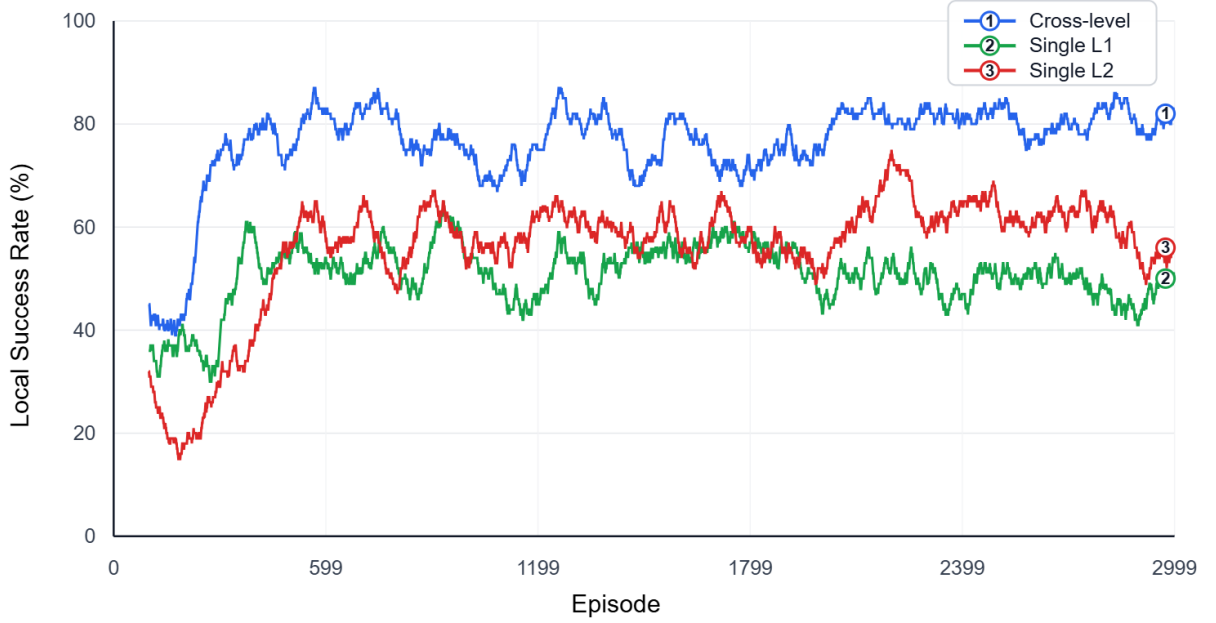


Figure 6.4: Local Success Rate over Training Episodes

To further understand how the single-level agents behave, we also evaluate their ability to stabilize the specific OT layer they protect. For each configuration, we compute a local success rate that measures how often the agent is able to drive the state variables of its own level back to the normal operating region within the allowed number of steps, regardless of what happens on the other level. The results on Figure 6.4 show that both single-level agents achieve noticeably higher local success rates than their global success rates: the Level 2 agent improves from roughly 50% global success to around 60% local success, and the Level 1 agent improves from about 20% to close to 50%. These findings indicate that single-level agents can be quite effective at mitigating disturbances within their own layer, but they often fail to prevent the attacker from maintaining or regaining a foothold in the rest of the system, reinforcing the need for the cross-level agent when end-to-end protection of the ICS is required.

Figure 6.5 summarizes at which attack stage each agent configuration is able to stop the adversary over the 3000 evaluation episodes. The x-axis corresponds to the eleven attack stages in our ICS kill chain (each mapped to a specific ATT&CK for ICS technique), and the y-axis reports the percentage of episodes in which the attack is successfully stopped at or before that stage for a given agent. The cross-level agent (blue bars) maintains a consistently high stopped-episode rate of around 80–90% across all stages, indicating that it can interrupt the intrusion both

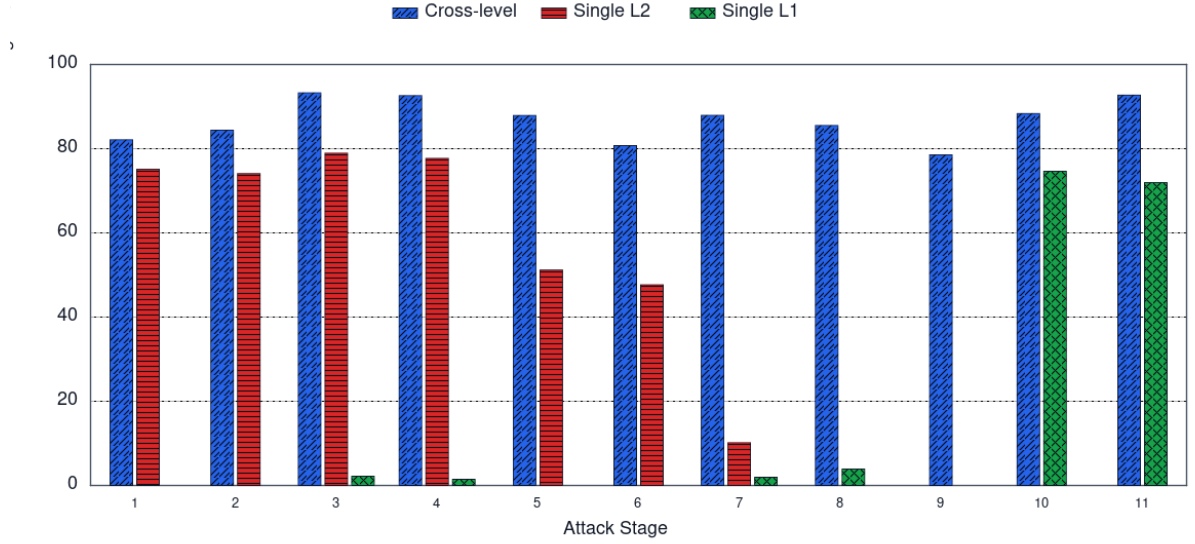


Figure 6.5: Attack Success Rate over Stage

in the early supervisory/network phases and during the later process-impact steps. In contrast, the Single L2 agent (red bars) performs well on the first stages, up to approximately Stage 5–6, where the attack is still primarily driven from Level 2; once the attacker has progressed to lateral movement and established command-and-control over Level 1 (after stage 6), the Level 2-only agent has little remaining leverage, and its ability to stop the attack drops sharply at later stages. The Single L1 agent (green bars) shows the opposite pattern: it is largely ineffective at the early and mid kill-chain stages, where most activity arises from Level 2, but its stopped-episode rate increases substantially for the final impact stages that directly manipulate the physical process, where Level 1 state changes become clearly visible. These results highlight that each single-level agent has a limited “window of opportunity” aligned with the stages that manifest most strongly on its own layer, whereas the cross-level agent can intervene effectively throughout the entire attack lifecycle.

Table 6.1 reports the average learned Q-values for each response action across the different ATT&CK for ICS techniques. For each technique, the action with the highest Q-value is highlighted as the agent’s preferred first response, but it is not the only action taken in practice—in most episodes it is followed by a short sequence of subsequent actions as the situation evolves. We observe that “Isolate device” becomes the best-Q action for three of the more severe techniques, yet its average Q-value remains moderate compared to less disruptive ac-

ID	Attack	Alert	Terminate SSH	Remove Attacker	Rate Limit	ARP Block	Disable Write	Disable User	Change Credentia	Block Read	Terminate HMI	Modify Physical	Isolate Device	Restore System
T0840	Network Connection Enumeration	3.03	2.57	2.48	2.83	2.51	2.62	2.59	2.59	-2.38	2.10	2.20	3.83	-0.54
T1110	Brute-force SSH	8.68	9.65	8.16	8.69	8.04	8.32	8.09	8.39	2.57	5.69	6.66	5.43	4.35
T0864	Transient cyber access (Login)	6.00	5.46	5.46	5.87	5.29	5.77	6.20	6.08	3.14	5.17	4.67	2.41	3.33
T0892	Change Credential	5.82	5.22	5.35	5.61	5.12	5.53	5.96	5.76	2.77	5.43	4.67	1.63	2.97
T0886	Exploit SSH Service Lateral Movement	9.24	10.27	8.91	9.02	8.51	8.71	8.32	8.25	3.38	6.87	6.05	5.89	6.86
T0830	Man in the Middle	3.99	3.57	3.34	3.68	3.59	3.53	3.51	3.54	-0.55	3.12	2.69	4.79	1.45
T0861	Point and Tag Identification	2.97	2.20	2.37	2.61	2.28	2.46	2.16	2.37	0.71	2.15	1.42	-0.73	0.57
T0836	Modify Parameter	3.92	3.46	3.28	3.73	3.41	3.97	3.37	3.63	2.89	3.33	6.13	-0.37	1.57
T0855	Unauthorized Command Message	4.04	3.57	3.48	3.87	3.69	3.91	3.53	3.63	2.85	3.79	6.27	-0.07	2.18
T0803	Block Command Message	4.65	4.34	4.55	5.05	5.02	4.78	4.78	4.90	4.68	4.60	4.55	4.00	5.30
T0814	Denial of Service	8.09	7.55	8.17	7.91	7.56	7.60	7.78	7.73	6.40	7.11	1.29	5.20	6.68
Avg Best Q-Value		5.58	5.26	5.05	5.35	5.00	5.20	5.12	5.17	2.41	4.49	4.24	2.91	3.16

Table 6.1: Q-Table Technique vs Actions

tions, reflecting the high penalty assigned to this critical response. More generally, the best-Q actions are distributed across several options, including raising an alert, terminating an SSH session, modifying physical parameters, and restoring the system state, rather than collapsing onto a single aggressive tactic. This pattern aligns with our reward design, which encourages actions that effectively stop attacks while avoiding unnecessary disruption by imposing strong penalties on heavy-weight interventions; as a result, the agent tends to avoid such critical actions in the early stages of an intrusion and only prioritizes them when the attack becomes more advanced or directly threatens the physical process. Overall, the learned policy appears reasonable and context-aware, selecting initial actions that match the severity and technique of each attack while still allowing follow-up actions when needed.

6.3 Detection Module Performance

To answer our second research question, we first compare a supervised transformer-based classifier with two large language models (LLMs) equipped with retrieval-augmented generation (RAG) for classifying text-based alerts into ATT&CK for ICS techniques. In the transformer setting, we fine-tune encoder-only models (DistilBERT and SecBERT) on our labeled alert corpus to perform multi-class sequence classification over the technique set $\mathcal{T}$. Each correlated and enriched alert is linearized into a single textual sequence and fed through the encoder

Model Type	Model	F1	Recall	Precision	Latency (ms)
Transformer	DistilBERT	0.9662	0.9685	0.9728	1.36
	SecBERT	0.9351	0.9346	0.9418	1.47
LLM	Meta-LLaMA 8B	0.9686	0.9623	0.9789	133
	Hermes	0.9839	0.9829	0.9852	183

Table 6.2: Performance comparison between Transformer-based and LLM-based classifiers

and a classification head, which directly outputs a probability distribution over techniques. In contrast, the LLM-based setting does not fine-tune the language model itself. Instead, we build a RAG pipeline that first retrieves the top- k candidate techniques (here, $k = 3$) and their textual descriptions from a curated ATT&CK for ICS / CTI index based on the alert text, and then prompts a Meta-LLaMA 8B or Hermes-2-Pro 8B model to select the most possible technique among these candidates. The LLMs thus rely on external knowledge and in-context reasoning rather than supervised gradient updates on our dataset.

Figure 6.6 and Table 6.2 summarize the macro-averaged F1, precision, and recall, together with the average inference latency per alert, for all four models. Both LLM-based approaches achieve slightly higher detection performance than the fine-tuned transformers: Hermes-2-Pro attains the best macro F1-score of approximately 0.984 with similar gains in precision and recall, while Meta-LLaMA 8B also marginally outperforms DistilBERT. DistilBERT, however, already reaches a macro F1 of around 0.967 with balanced precision and recall, and SecBERT trails it by a small margin. The main difference lies in inference cost. On our evaluation hardware, processing a single alert takes roughly 1.3–1.5 ms for the transformer encoders, but around 133 ms for Meta-LLaMA 8B and 183 ms for Hermes-2-Pro. In other words, the LLMs are two orders of magnitude slower than the fine-tuned transformers for only a modest improvement in classification accuracy.

Given the strict timing and availability constraints of OT environments, where detection models must operate continuously and support near real-time response on resource-constrained infrastructure, this trade-off is critical. Our results suggest that fine-tuned transformer encoders such as DistilBERT provide a more practical balance between technique-classification quality and latency for online ICS detection pipelines, while LLM+RAG configurations are better

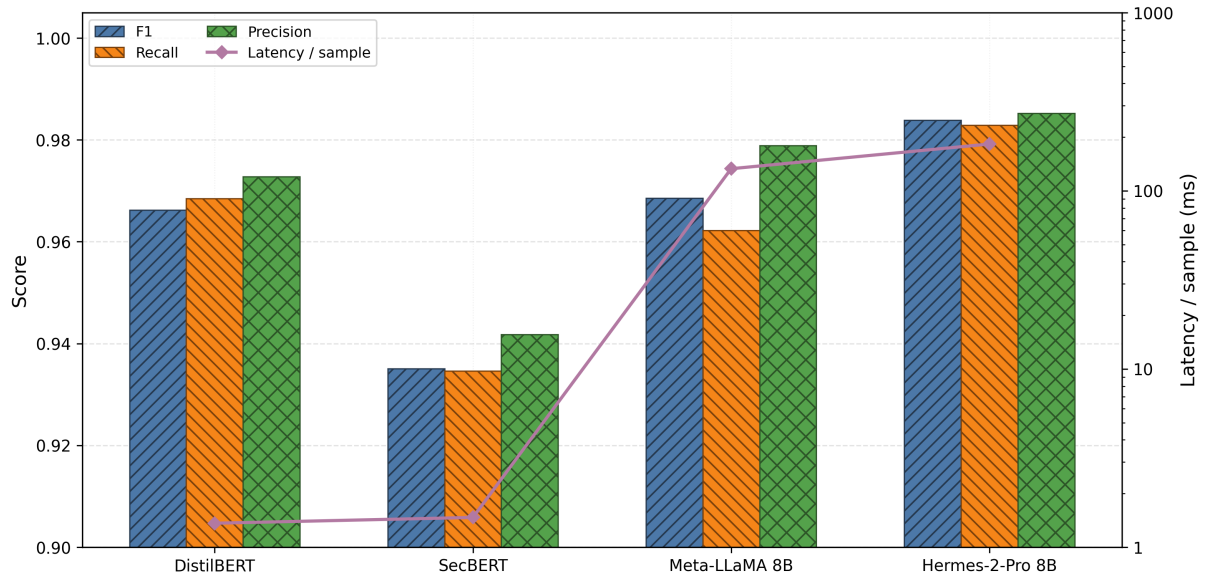


Figure 6.6: Transformer vs LLM Performance Comparison

suited for offline analysis or analyst-assistance scenarios where higher latency is acceptable in exchange for slightly richer reasoning capability.

Figure 6.7 compares the confusion matrices of the unsupervised learning detector under different input-source settings. The multi-source fusion setting (logs + sensors + traffic) achieves the best attack detection performance, with an attack recall of 99.96%, indicating that combining heterogeneous sources substantially reduces false negatives. This is because the alarm is raised when evidence from any source appears within the observation window, making attacks less likely to be missed. In contrast, single-source settings show a clear performance gap, with attack recall dropping to 77.51% for logs, 40.96% for traffic, and 36.06% for sensors. Among them, logs provide the strongest single-source performance, suggesting that they are the most informative modality for attack detection and that many attacks are primarily captured through endpoint-level log evidence. Traffic and sensor streams, by comparison, often reflect only partial or downstream effects of the attack. This also explains the trend observed in the previous figure 6.2: removing either L1 logs or L2 logs leads to a noticeable drop in overall performance, further confirming the central role of logs in the fusion model. Despite these differences in attack detection, all settings maintain high recognition of normal behavior, with normal-class accuracy remaining consistently high, showing that the unsupervised detector can learn the normal operational baseline effectively.

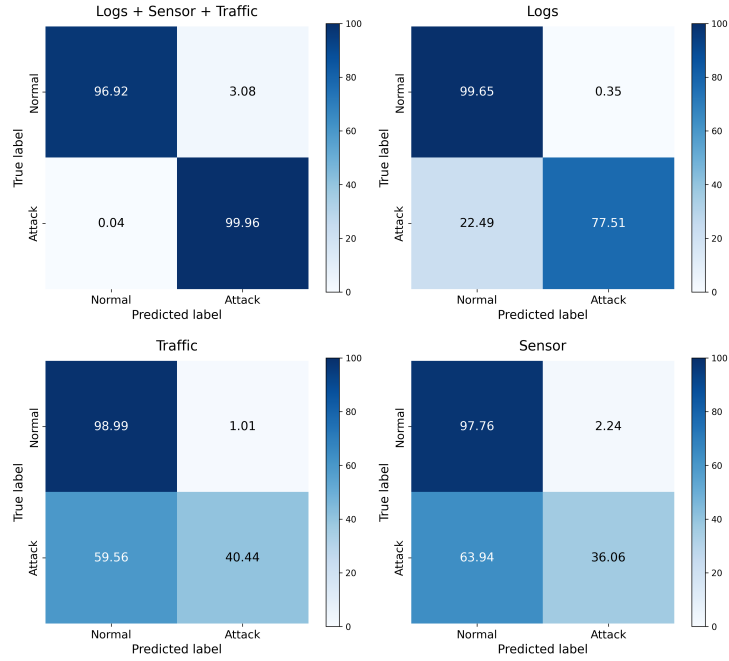


Figure 6.7: Confusion Matrices of the Anomaly Detection Module Across Multi-Source

6.4 State Space: Big vs Small

To assess the impact of the state definition, we compare two configurations of the cross-level agent: a *small-state* variant that observes only detection-related signals (e.g., predicted technique labels and alert indicators), and a *big-state* variant that additionally includes selected physical process variables and safety flags (tank levels, overflow indicators, unreachable/isolated device flags, etc.). Figure 6.8 shows the evolution of the global success rate over 3000 training episodes. The small-state agent converges more quickly in the early phase (within roughly 500 episodes) due to its lower-dimensional state space, but its asymptotic success rate remains consistently below that of the big-state agent and exhibits a less stable learning trend. In contrast, the big-state agent learns more slowly at first, yet eventually achieves a higher and more stable success rate. A similar pattern appears in the reward curves in Figure 6.9: although both agents reach positive returns, the big-state configuration attains a slightly higher plateau with reduced variance. These results indicate that exposing the agent to additional physical state information not only slows down initial learning but also enables more accurate action selection in the long run, yielding a more robust response policy.

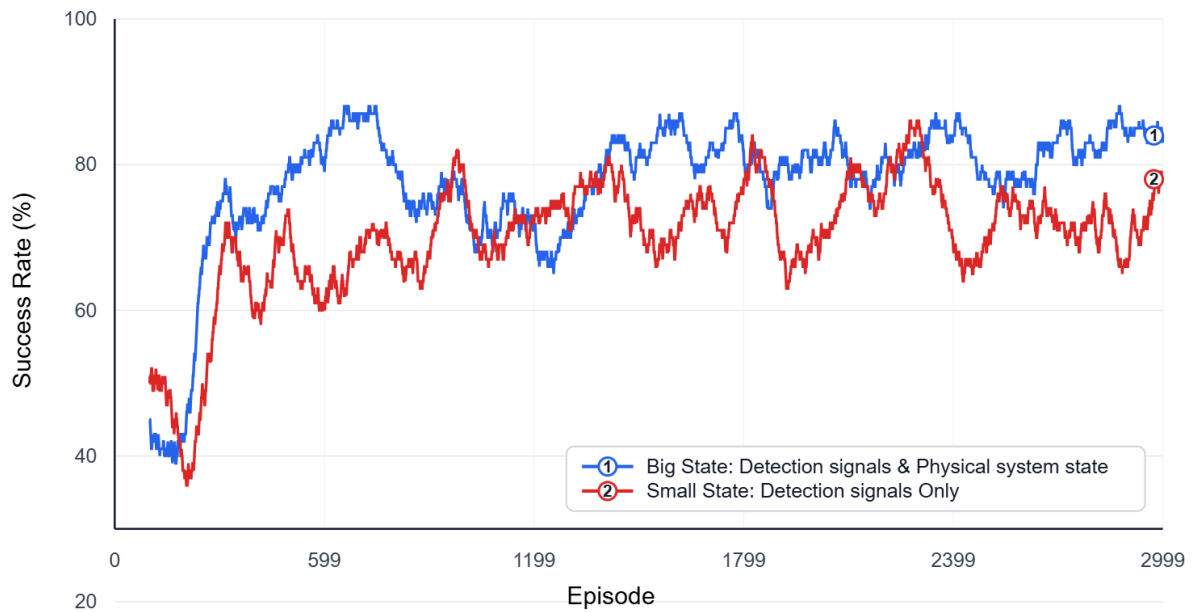


Figure 6.8: Success Rate Comparison: Full State vs Small State

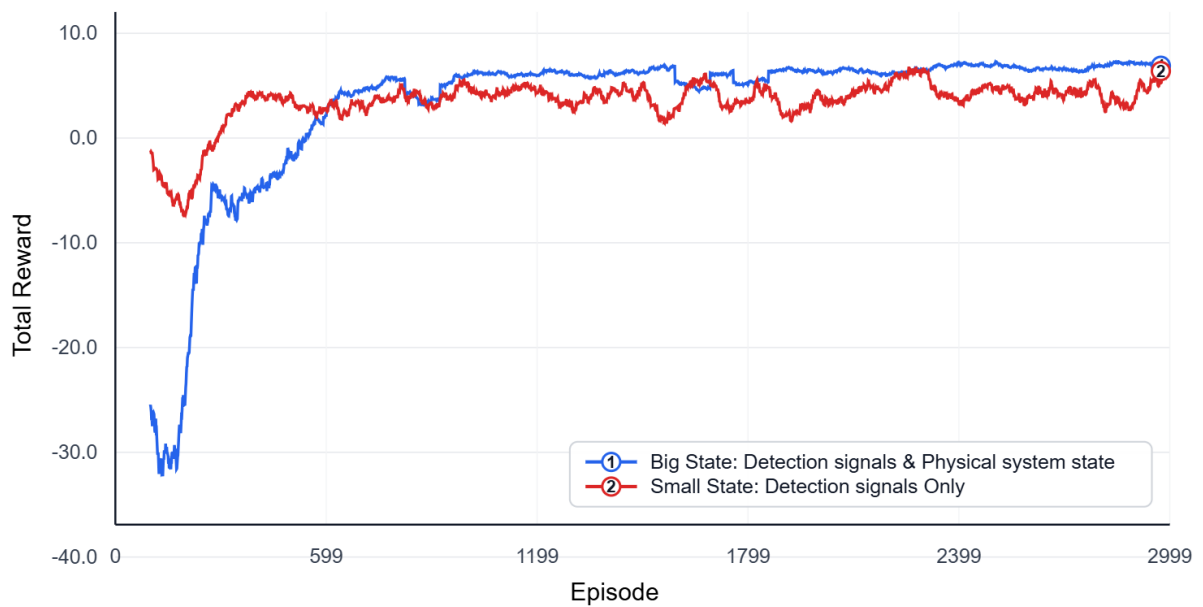


Figure 6.9: Reward Curve Comparison: Full State vs Small State

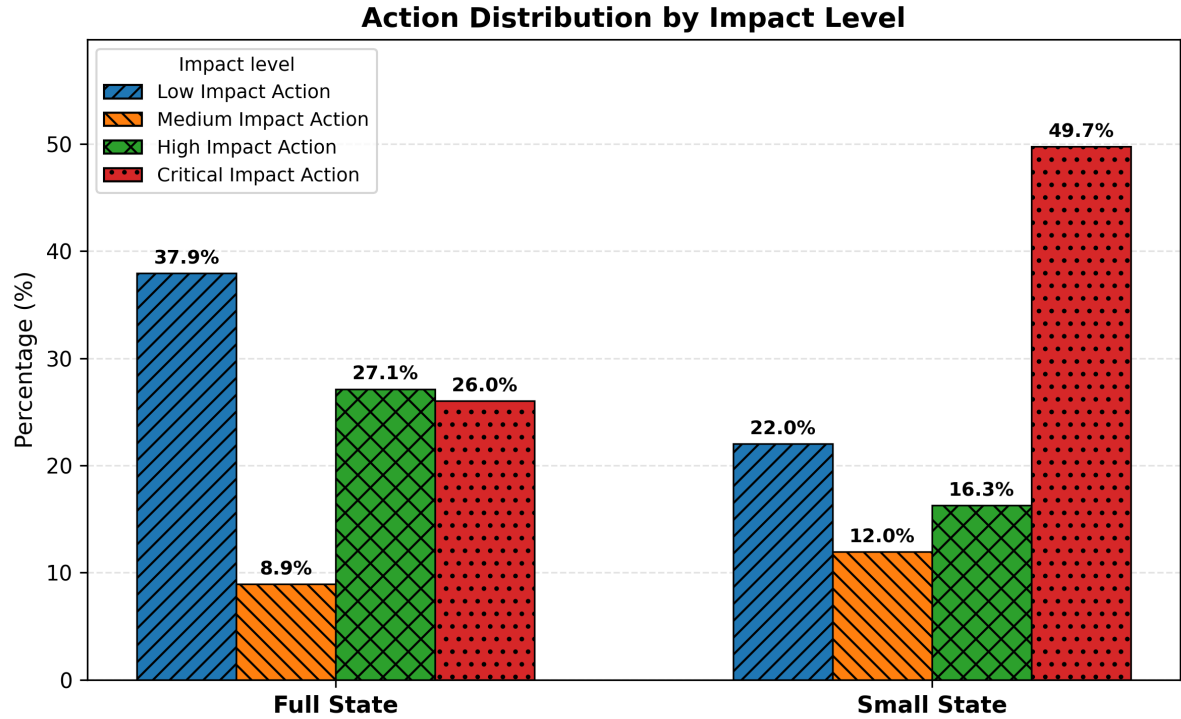


Figure 6.10: Action Distribution Comparison: Full State vs Small State

Figure 6.10 further analyzes how the state definition influences the type of response actions chosen by the agent. We group actions into four impact levels (low, medium, high, and critical) based on how disruptive they are to the OT process, with operations such as device isolation or full state restoration treated as critical interventions. For the small-state agent, nearly half of all selected actions fall into the critical-impact category, while low-impact actions account for only around one fifth of decisions. This indicates a tendency to over-react: without access to the underlying physical state, the agent often resorts to heavy-weight responses that have a higher probability of suppressing anomalies but may be unnecessary from a process-safety viewpoint. In contrast, the big-state agent predominantly chooses low-impact actions and uses high- and critical-impact actions much more sparingly. Because it can observe whether the physical process remains within safe bounds or is approaching dangerous conditions, the big-state agent can keep its responses conservative when the system is stable and escalate only when physical variables signal genuine risk. Taken together, these findings suggest that incorporating carefully designed physical state features is essential for avoiding over-reaction in OT environments while still maintaining strong overall mitigation performance.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis proposed and evaluated a cross-level AI defense framework for industrial control systems that jointly addresses intrusion detection and automated response. The system combines an unsupervised learning and explainable AI module that detects anomalies from multi-level telemetry and generates anomaly alerts, a transformer-based classifier that maps these alerts to MITRE ICS techniques, and a Deep Q-Learning agent that selects response actions across both levels. All experiments are conducted on a testbed that emulates an ICS/OT water treatment plant with scripted multi-stage attack campaigns, providing a realistic environment to assess the end-to-end behavior of the proposed framework.

The first research question asked whether a cross-level agent can detect and stop attacks more effectively than single-level agents restricted to either Level 1 or Level 2. At the detection stage, the cross-level configuration achieves substantially higher macro F1, precision, and recall than both single-level variants and remains robust across all techniques. In contrast, the Single-level L1 and L2 agents show larger class-specific errors, especially on techniques with similar patterns or cross-layer behaviours, where they frequently confuse pairs such as T0886 and T1110 or T0836 and T0855. At the response stage, evaluated over 3000 attack episodes, the cross-level agent converges to a high and stable attack-blocking success rate, while the Single-level L2 agent remains clearly lower and the Single-level L1 agent performs worst. These results indicate that single level agent do not guarantee robustness: without cross-level context, single-level agents misclassify techniques that share local patterns and then select sub-optimal actions. Cross-level visibility and control are therefore essential to reliably distinguish similar techniques and contain attack chains that span multiple ICS layers.

The second research question compared transformer-based and LLM-based models for text-based alert classification. In our experiments, a lightweight transformer (DistilBERT) already achieves macro F1, precision, and recall around 0.97 with an average latency of about 1.4 ms per alert, while LLM-based classifiers provide only modest additional accuracy. However, these LLM models are much slower, with latencies in the range of 130—180 ms per alert. This suggests that transformer-based models are the natural choice for the main detection pipeline, whereas LLM-based classifiers are better suited for post-incident analysis or targeted investigations where a small accuracy gain can justify the higher cost.

The third research question examined how the definition of the reinforcement learning state, in particular the inclusion of physical process variables, influences the learned response policy and the risk of over-reaction. A compact state that encodes only detection-related information converges faster but leads the agent to overuse high- and critical-impact actions such as device isolation and full system restoration. When the state is extended with process measurements and safety flags, learning is slower in the early episodes but ultimately reaches similar or higher attack-blocking success and greater cumulative reward. At the same time, the agent learns to use low- and medium-impact actions that are sufficient to stop attacks and only applies critical interventions when they are strictly necessary.

7.2 Future Work

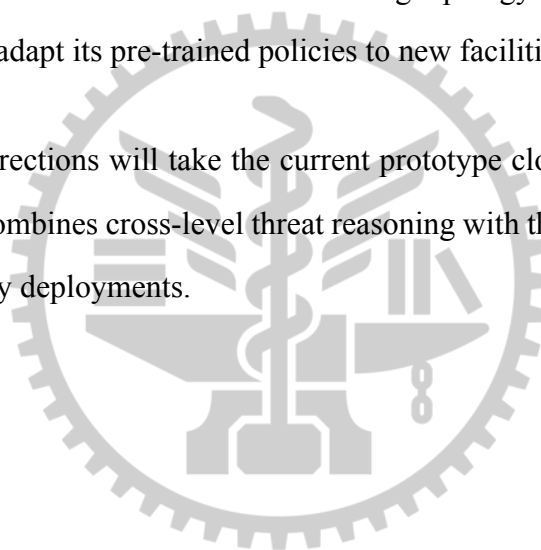
Several extensions can further strengthen the robustness of the proposed cross-level framework. First, the current evaluation still uses a limited set of scripted attack campaigns. Future work should build stronger and more diverse attack scenarios, for example longer multi-stage sequences or coordinated attacks that involve multiple devices at the same time. These additional tests would give a clearer picture of how the system behaves under more realistic and adaptive attackers instead of fixed, pre-defined scripts.

Second, the current prototype is using a Single Agent Architecture that observes and controls both Level 1 and Level 2. To better align with the distributed nature of industrial environments,

a promising future direction is transforming to a Multi-Agent Learning (MARL) architecture. By partitioning the large-scale system into manageable segments based on operational zones or hierarchical levels, dedicated sub-agents can manage specific tasks while coordinating either through peer-to-peer communication or a central supervisory agent, substantially improving the framework's scalability and fault tolerance.

Finally, an important direction is to investigate how well the learned detection models and response policies generalise across different industrial sites. The current results are obtained on a single testbed with a fixed topology and process. Future studies should evaluate the system across diverse ICS environments by leveraging Transfer Learning and Domain Adaptation techniques. Specifically, research could focus on extracting topology-agnostic state representations, enabling the Agent to adapt its pre-trained policies to new facilities with minimal fine-tuning.

Addressing these directions will take the current prototype closer to an operational defense framework, one that combines cross-level threat reasoning with the robustness required for live industrial cybersecurity deployments.



References

- [1] Sangfor Technologies. (2025) Understanding the purdue model for ics & ot security. [Online]. Available: <https://www.sangfor.com/glossary/cybersecurity/what-is-purdue-model-ics-security>
- [2] Y.-W. Ma, Y.-H. Tu, C.-W. Tsou, Y.-N. Chiang, and J.-L. Chen, “A survey of cyber security and safety in industrial control systems,” *Journal of Internet Technology*, vol. 25, no. 4, pp. 541–550, July 2024. [Online]. Available: <https://jit.ndhu.edu.tw/article/view/3071>
- [3] The MITRE Corporation, “ICS Matrix | MITRE ATT&CK®,” 2022. [Online]. Available: <https://attack.mitre.org/matrices/ics/>
- [4] National Institute of Standards and Technology, “The nist cybersecurity framework (csf) 2.0,” NIST, Gaithersburg, MD, Cybersecurity Framework, 2024, nIST Cybersecurity Framework Version 2.0. [Online]. Available: <https://www.nist.gov/cyberframework>
- [5] Lee, J. H. and Ji, I. H. and Jeon, S. H. and Seo, J. T., “Anomaly Detection Method Considering PLC Control Logic Structure for ICS Cyber Threat Detection,” *Applied Sciences*, vol. 15, no. 7, p. 3507, Apr. 2025.
- [6] M. Alikhani and R. Kazemi, “Contrastive-kan: A semi-supervised intrusion detection framework for cybersecurity with scarce labeled data,” *arXiv preprint arXiv:2507.10808*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.10808>
- [7] Berge, V. and Li, C., “Enhanced Anomaly Detection in Industrial Control Systems aided by Machine Learning,” *arXiv preprint arXiv:2410.19717*, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.19717>
- [8] Fortinet, Inc., “2024 state of operational technology and cybersecurity report,” Fortinet, Sunnyvale, CA, Tech. Rep., 2024. [Online]. Available: <https://www.fortinet.com/content/dam/fortinet/assets/reports/report-state-ot-cybersecurity.pdf>

- [9] Reaney, Matthew and McLaughlin, Kieran and Grant, James, “Network Intrusion Response using Deep Reinforcement Learning in an Aircraft IT-OT Scenario,” in *Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES 2024)*. Vienna, Austria: ACM, July 2024, pp. 1–7.
- [10] Yan, Zhang and Shukla, Piyush Kumar and Shukla, Prashant Kumar and Thakur, Kanika and Sinha, Anurag and Khalid, Saifullah, “Intrusion Detection and Mitigation Method for the Industrial Internet of Things Using Bidirectional Convolutional Long Short-Term Memory and Deep Recurrent Convolutional Q-Networks,” *International Journal of Information Security*, 2025.
- [11] Mem, J. and Hatch, K. and Silva, R. and Hickert, C. and Sookoor, T. and Kochenderfer, M. J., “Autonomous Attack Mitigation for Industrial Control Systems,” in *Proceedings of the 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*. IEEE, Jun. 2022.
- [12] Chen et al., “Interpretable Cross-Level Intrusion Response System Based on Deep Reinforcement Learning for Industrial Control Systems,” *IEEE Transactions on Industrial Informatics*, 2024.
- [13] Sun, S. et al., “A Reinforcement Learning Engine with Reduced Action and State Space for Cyber-Physical Optimal Response,” *arXiv preprint arXiv:2410.04518*, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.04518>
- [14] Radoglou-Grammatikis, K. S. and Sarigiannidis, P. and Mavropoulos, A., “Defending Industrial Internet of Things Against Modbus/TCP Threats: A Combined AI-Based Detection and SDN-Based Mitigation Solution,” *International Journal of Information Security*, vol. 24, 2025.
- [15] Alqithami, S., “Hierarchical Adversarially-Resilient Multi-Agent Reinforcement Learning for Cyber-Physical Systems Security,” *arXiv preprint arXiv:2506.22445*, Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.22445>

- [16] U.S. Department of Energy, “Purdue Model Framework for Industrial Control Systems & Cybersecurity Segmentation,” Infrastructure Security and Energy Restoration Division, Tech. Rep., 2019.
- [17] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, “Guide to industrial control systems (ics) security,” National Institute of Standards and Technology, NIST Special Publication 800-82 Revision 2, May 2015. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-82r2>
- [18] A. M. Y. Koay, R. K. L. Ko, H. Hettema, T. Holz, B. Stock, and M. Antal, “Machine learning in industrial control system (ics) security: current landscape, opportunities and challenges,” *Journal of Intelligent Information Systems*, vol. 60, no. 2, pp. 377–405, 2023. [Online]. Available: <https://doi.org/10.1007/s10844-022-00753-1>
- [19] M. A. Umer, K. N. Junejo, M. T. Jilani, and A. P. Mathur, “Machine learning for intrusion detection in industrial control systems: Applications, challenges, and recommendations,” *International Journal of Critical Infrastructure Protection*, vol. 38, p. 100516, 2022.
- [20] C. Fung, E. Zeng, and L. Bauer, “Attributions for ML-based ICS anomaly detection: From theory to practice,” in *Proceedings of the 2024 Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2024.
- [21] D. T. Ha, N. X. Hoang, N. V. Hoang, N. H. Du, T. T. Huong, and K. P. Tran, “Explainable anomaly detection for industrial control system cybersecurity,” *IFAC-PapersOnLine*, vol. 55, no. 13, 2022.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [23] H. Kheddar, “Transformers and large language models for efficient intrusion detection systems: A comprehensive survey,” *Information Fusion*, 2025, preprint available as arXiv:2408.07583.

- [24] O. Dogru, J. Xie, O. Prakash, R. Chiplunkar, J. Soesanto, H. Chen, K. Velswamy, F. Ibrahim, and B. Huang, "Reinforcement learning in process industries: Review and perspective," *IEEE/CAA Journal of Automatica Sinica*, vol. 11, no. 2, pp. 283–300, Feb. 2024.
- [25] M. Ghasemi, A. H. Moosavi, I. Sorkhoh, A. Agarwal, F. Alzhouri, and D. Ebrahimi, "An introduction to reinforcement learning: Fundamental concepts and practical applications," *arXiv preprint arXiv:2408.07712*, Aug. 2024. [Online]. Available: <https://arxiv.org/abs/2408.07712>
- [26] J. Mern, K. Hatch, R. Silva, J. Brush, and M. J. Kochenderfer, "Reinforcement learning for industrial control network cyber security orchestration," *arXiv preprint arXiv:2106.05332*, 2021.
- [27] A. Dehlaghi-Ghadim, A. Balador, M. H. Moghadam, H. Hansson, and M. Conti, "ICSSIM — a framework for building industrial control systems security testbeds," *Computers in Industry*, vol. 148, p. 103906, 2023.
- [28] "Docker container manual," <https://docs.docker.com/manuals/>, accessed: 2026-4-27.
- [29] MITRE Corporation, "Mitre caldera™: Automated adversary emulation platform," <https://github.com/mitre/caldera>, accessed: 2026.
- [30] A. Dehlaghi-Ghadim, "Icsflow," <https://github.com/AlirezaDehlaghi/ICSFlow>, 2022, gitHub repository.
- [31] "Wireshark: A network traffic analyzer," <https://github.com/wireshark/wireshark>, accessed: 2026.
- [32] "Rsyslog: A rocket-fast system for log processing," <https://github.com/rsyslog/rsyslog>, accessed: 2026.
- [33] "Tcpdump: The classic command-line packet analyzer," <https://www.tcpdump.org/>, accessed: 2026.

- [34] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [35] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [36] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. E. Gonzalez, M. I. Jordan, and I. Stoica, “Ray RLlib: A composable and scalable reinforcement learning library,” *arXiv preprint arXiv:1712.09381*, 2017. [Online]. Available: <https://arxiv.org/abs/1712.09381>
- [37] “Matplotlib: Plotting with python,” <https://github.com/matplotlib/matplotlib>, accessed: 2026.
- [38] “Pandas: Powerful python data analysis toolkit,” <https://github.com/pandas-dev/pandas>, accessed: 2026.
- [39] “Seaborn: Statistical data visualization in python,” <https://github.com/mwaskom/seaborn>, accessed: 2026.
- [40] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

- [41] R. Řehůřek and P. Sojka, “Software framework for topic modelling with large corpora,” in *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. Valletta, Malta: ELRA, 2010, pp. 45–50.

